

MOBILE OBJECT DETECTION IN MACHINE LEARNING

A PROJECT REPORT

Submitted

In partial fulfillment of the requirements for the award of the degree of

MASTER OF SCIENCE

IN INFORMATION TECHNOLOGY

MSc IT

By

KAKOOZA TOBIAS

Roll No. S20/0166

**FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
INTERNATIONAL BUSINESS OF SCIENCE AND TECHNOLOGY UNIVERSITY**



ISBAT University

Plot 11A, Rotary Avenue, Lugogo By-Pass, Kampala

SEPTEMBER 2021



**ISBAT
UNIVERSITY**
A CHARTERED UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report titled **MOBILE OBJECT DETECTION IN MACHINE LEARNING** bonafide work of **KAKOOZA TOBIAS** who carried out the project work under my supervision

DEAN FACULTY OF ICT & ENGINEERING

Dr. Tyagi Vaibhav Bhushan, Phd, MTech
Associate Dean/ Associate Professor

SIGNATURE

Date

SUPERVISOR

Dr. Vivekananth Padmanabhan, Phd

SIGNATURE

Date



**ISBAT
UNIVERSITY**
A CHARTERED UNIVERSITY

ISBAT University Kampala

DECLARATION

I **KAKOOZATOBIAS, S20/0166** declare that this Project Report is an original work and that has never been submitted to any institution for any award

Date:

Signed:

ACKNOWLEDGEMENT

I am overwhelmed in all humbleness and gratefulness to acknowledge my depth to all those who have helped me to put these ideas, well above the level of simplicity and into something concrete.

I would like to express my special thanks of gratitude to my lecturers as well as our coordinator Dr. Vivek who gave me the golden opportunity to do this wonderful project on the topic (Object Detection in Machine Learning), which also helped me in doing a lot of Research and I came to know about so many new things. I am really thankful to them.

Any attempt at any level cannot be satisfactorily completed without the support and guidance of my friends especially Mr. Kalyango Mark, Mr. Olam Anthony and Mrs. Kyeyago Flavia who inspired and motivated me to continue with the Masters' program.

I would like to once again thank my supervisor and lecturers for their excellent guidance and support during the process. I have been given tips and pointers and good feedback throughout this period.

Finally, I would like to thank my family for their support and encouragement during this period.

ABSTRACT

Object Detecting and tracking objects are among the most prevalent and challenging tasks that a surveillance system has to accomplish in order to determine meaningful events and suspicious activities, and automatically annotate and retrieve video content. Under the business intelligence notion, an object can be a face, a head, a human, a queue of people, a crowd as well as a product on an assembly line. In this report, we introduce the reader to main trends and provide taxonomy of popular methods to give an insight to underlying ideas as well as to show their limitations in the hopes of facilitating integration of object detection and tracking for more effective business oriented video analytics.

This development has enabled engineers to replace existing heuristics based systems in favour of machine learning models with superior performance. In this report, we evaluate the viability of using deep learning models for object detection in real-time video feeds on mobile devices in terms of object detection performance and inference delay as either an end-to-end system or feature extractor for existing algorithms. Our results show a significant increase in object detection performance in comparison to existing algorithms with the use of transfer learning on neural networks adapted for mobile use.

Table of Contents

BONAFIDE CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v

Chapter 1

1.0 INTRODUCTION	1
1.1 Problem statement	2
1.2 Project title (name)	2
1.3 Project Definition	3
1.4 Project description	3
1.5 Purpose	3
1.6 Scope	3
1.4 Project scope	3
1.4 Objectives of the system	3
1.5 Project scope	3
1.6 Existing system and its advantages	4
1.7 Proposed system	4
1.7.1 Advantage of proposed system	4
1.8 Hardware Requirements	5
1.9 Software Requirements	5

Chapter 2

2.0 SYSTEM ANALYSIS	6
2.1 Literature Survey	6
2.2 Existing system	7

2.3	Proposed system	8
2.4	Feasibility study	9

Chapter 3

3.0.1	SYSTEMS DESIGN	10
3.0.2	Module description	10
3.0.3	Model Customization.....	13
3.11	Work flow	15
3.12	How object detection can be performed in tensorflow	16
3.13	Dataset Sample view	17
3.14	Flowchat	20

Chapter 4

4.11	System development	22
4.121	Screenshots	22
4.122	Coding.....	34

Chapter 5

5.01	Conclusion	119
5.02	Summary	121
5.03	Future Work	122
5.04	References	123

CHAPTER 1

INTRODUCTION

1. Introduction

Humans can easily detect and identify objects present in an image. The human visual system is fast and accurate and can perform complex tasks like identifying multiple objects and detect obstacles with little conscious thought. With the availability of large amounts of data, faster GPUs, and better algorithms, we can now easily train computers to detect and classify multiple objects within an image with high accuracy.

1.0 Problem Statement

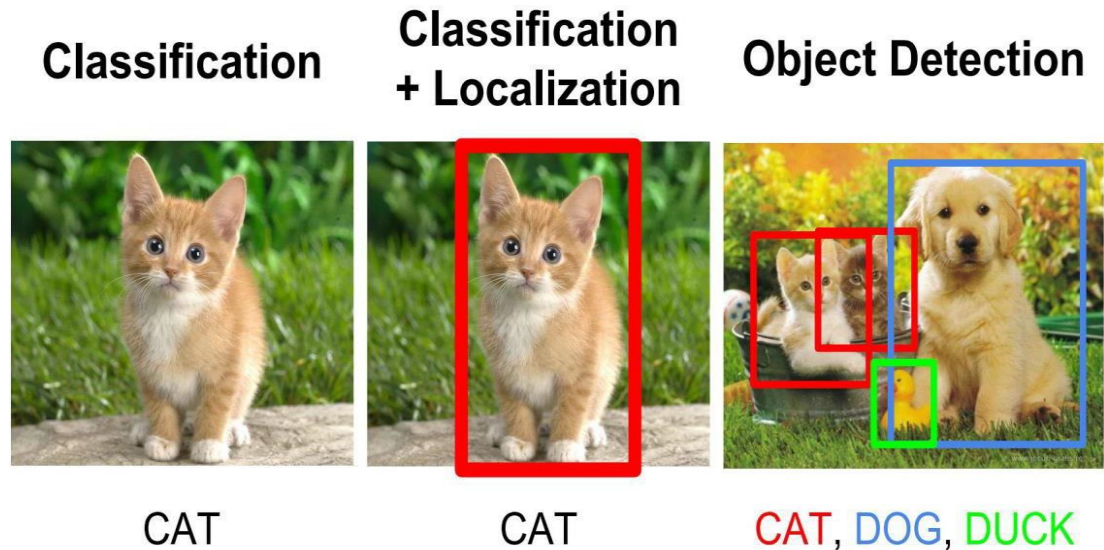
As we move towards more complete image understanding, having more precise and detailed object recognition becomes crucial. In this context, one cares not only about classifying images, but also about precisely estimating the class and location of objects contained within the images, a problem known as object detection. One of the major problem was that of image classification, which is defined as predicting the class of the image. A slightly complicated problem is that of image localization, where the image contains a single object and the system should predict the class of the location of the object in the image (a bounding box around the object). The more complicated problem (this project), of object detection involves both classification and localization.

With the above observation, a need for object detectors in surveillance such as security cameras to identify criminals who trespass or want to commit crime, a need for object detectors in Medicine such as a task of identifying medical-based objects within an image, a need to help visually impaired people to Identify objects around them though voice detection and many others will be possible with this application.

It has the following objectives:

- Object detection, segmentation, location, and recognition
- Object tracking
- Different perspectives on the same scene or object based image retrieval

- Medical-object detection



Process of Image Detection in Computer Vision

The image above shows the process of image detection in computer vision. In the figure, the first step is video input. The next steps involve the application of image processing techniques and extraction of the object. Following this, image recognition takes place. Finally, a description **Object Detection** of the object is obtained. In this process, image processing is a very important step. It involves converting the original image data to digital data for the use of the computer.

Application

A well known application of object detection is face detection, that is used in almost all the mobile cameras. A more generalized (multi-class) application can be used in autonomous driving where a variety of objects need to be detected. Also it has a important role to play in surveillance systems. These systems can be integrated with other tasks such as pose estimation where the _rst stage in the pipeline is to detect the object, and then the second stage will be to estimate pose in the detected region. It can be used for tracking objects and thus can be used in robotics and medical applications. Thus this problem serves a multitude of applications

1.1 Project title

MOBILE OBJECT DETECTION IN MACHINE LEARNING

1.2 PROJECT DEFINITION

Object detection is a computer vision technique for locating instances of objects in images or videos. Object detection algorithms typically leverage machine learning or deep learning to produce meaningful results.

1.3 PROJECT DESCRIPTION

An object detection model is trained to detect the presence and location of multiple classes of objects. For example, a model might be trained with images that contain various pieces of fruit, along with a label that specifies the class of fruit they represent (e.g. an apple, a banana, or a strawberry), and data specifying where each object appears in the image.

When an image is subsequently provided to the model, it will output a list of the objects it detects, the location of a bounding box that contains each object, and a score that indicates the confidence that detection was correct.

1.4 PURPOSE

Object detection is inextricably linked to other similar computer vision techniques like image recognition and image segmentation, in that it helps us understand and analyze scenes in images or video.

1.5 SCOPE

Deep learning-based object detection models typically have two parts. An encoder takes an image as input and runs it through a series of blocks and layers that learn to extract statistical features used to locate and label objects. Outputs from the encoder are then passed to a decoder, which predicts bounding boxes and labels for each object.

The simplest decoder is a pure regressor. The regressor is connected to the output of the encoder and predicts the location and size of each bounding box directly. The output of the model is the X, Y coordinate pair for the object and its extent in the image. Though simple, this type of model is limited. You need to specify the number of boxes ahead of time. If your image has two dogs, but your model was only designed to detect a single

object, one will go unlabeled. However, if you know the number of objects you need to predict in each image ahead of time, pure regressor-based models may be a good option.

1.6 Existing System

- Early implementations of object detection involved the use of classical algorithms, like the ones supported in OpenCV, the popular computer vision library. However, these classical algorithms could not achieve enough performance to work under different conditions.
- Early object detectors were based on handcrafted features.
- Sliding window classifier, check if feature response is strong enough, if so: output detection.
- We will review a few representative features pre-deep learning.
 - ✓ Haar-like features such as linear, edge to get an output.
 - ✓ Histograms of Oriented Gradients.

1.7 Proposed System

In the proposed generation, there is a preprocessing technique that has been widely used in current object detection pipelines to guide the search of objects and avoid exhaustive sliding window search across images.

The proposed system has an improved algorithm based on target proposal and one-stage target detection to improve the target detection performance.

1.7.1 Advantage of proposed system

Object detection is inextricably linked to other similar computer vision techniques like image recognition and image segmentation, in that it helps us understand and analyze scenes in images or video.

1.8 Hardware Requirements

I3

4 GB RAM

15” Color Monitor

Keyboard

Mouse

1.9 Software Requirements

Operating System : Windows 10

Front End Software : Android Studio;Xml,tensorflow

Back End Software : Android Studio; Keras,tensorflow

CHAPTER 2

2.0 SYSTEM ANALYSIS

Object detection is one of the most basic and central task in computer vision. Its task is to find all the interested objects in the image, and determine the category and location of the objects. Object detection is widely used and has strong practical value and research prospects

2.1 Literature Survey

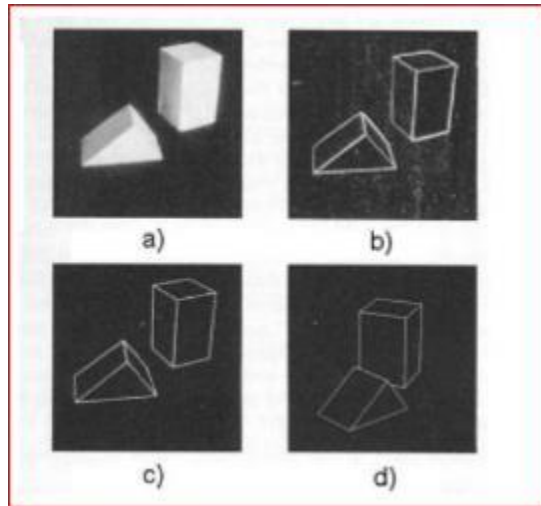
In past few years, the detection of Objects in real time and Image processing has become an active area of research and several new approaches have been proposed.

Several researchers have conducted many studies about Object detection-

- S.V. Viraktamath, Mukund Katti, Aditya Khatawkar & Pavan Kulkarni has conducted a study of openCV and also have published an IEEE paper for Face Detection and Tracking using OpenCV. Their work is related with converting web cam captured 2D Images and convert them into 3D Images related to human faces by constructing 3D Geometry data outputs.
- A.Rodriguez used HSV (Hue, Saturation, Value) model for more accurate processing of image. For training and recognizing image ANN (Artificial Neural Network) algorithm is used. To separate foreground and background from image sobel edge detection is used.
- J. A. Hesch, S.I. Roumeliotis also provide an idea to implement ORB(Oriented FAST and Rotated BRIEF(Binary Robust Independent Elementary Features) algorithm to increase the execution speed by utilizing the reconfigurable nature and pipeline. ORB algorithm builds on the well-known FAST key point detector and the recently developed BRIEF descriptor.
- TensorFlow is an open source framework developed by Google researchers to run machine learning, deep learning and other statistical and predictive analytics workloads.
-

2.2 Existing System

- Early implementations of object detection involved the use of classical algorithms, like the ones supported in OpenCV, the popular computer vision library. However, these classical algorithms could not achieve enough performance to work under different conditions.
- It is important to check how computer vision research advanced, and notice the change in the way of thinking about visual problems. There are three major approaches to object recognition.



- a) A blocks world scene
- b) Detected edges
- c) A 3D polyhedral description of the scene
- d) The 3D scene displayed from a different viewpoint

✓ Geometry-based approaches: This was the earliest approach, and used geo-metric models to account for the variation in an object's appearance due to viewpoint, illumination and occlusion changes. The principle was that the geometric description of a 3D object would allow for an accurate prediction of the projected shape in a 2D image, as shown in Figure above. Much attention was made to the extraction of geometric primitives that are invariant to view-point changes at the time, and many algorithms were developed, for different representations of the world from the blocks world, to the world of generalized cylinders. These approaches suffered from the difficulty of extracting primitives, the computational

complexity for certain shapes, noisy geometry, missing features, etc.

- ✓ Appearance-based approaches: At the end of the geometric era, scientists started discovering appearance-based techniques and a lot of interest was generated around them. Most notably, the eigen face methods attracted a lot of attention as it was one of the very first face recognition systems that was relatively accurate and not so computationally complex making it more efficient, like the work of Turk
- ✓ Feature-based approaches: In feature-based object detection, standardization of image features and registration (alignment) of reference points are important. The images may need to be transformed to another space for handling changes in illumination, size and orientation. One or more features are extracted and the objects of interest are modelled in terms of these features. Object detection and recognition then can be transformed into a graph matching problem
 - Early object detectors were based on handcrafted features.
 - Sliding window classifier, check if feature response is strong enough, if so: output detection.
 - We will review a few representative features pre-deep learning.
 - ✓ Haar-like features such as linear, edge to get an output.

2.3 Proposed System

In the proposed generation, there is a preprocessing technique that has been widely used in current object detection pipelines to guide the search of objects and avoid exhaustive sliding window search across images.

The proposed system has an improved algorithm based on target proposal and one-stage target detection to improve the target detection performance.

2.4 Feasibility Study

The objective of object detection is to localize and classify semantic objects in images, as seen in figure the. Object detection has been widely used for vehicle detection, face recognition, pedestrian counting, autonomously driving cars and security systems.

As object detection methods are a suitable method for real-time inference, we applied object detection in our research. The main goal in object detection is usually to enclose objects in a bounding box and to predict the class of the object in the bounding box.

Two types of state-of-the-art object detection networks can be identified. On the one hand, we have two-stage object detection networks. Examples are Faster R-CNN (Region based Convolutional Neural Networks) and Mask R-CNN

On the other hand, we have one-stage object detection networks. One-stage object detection networks are often faster but are often less accurate. As a low inference time is an important factor for our research, we focus on one-stage object detection. We only briefly examine two-stage object detection networks. Generally, two-stage object detection networks have a higher inference time and are therefore less suitable for real-time prediction. They often are also less straightforward to implement and train.

CHAPTER 3

3.01 SYSTEM DESIGN

In the system design, An object detection model is trained to detect the presence and location of multiple classes of objects. For example, a model might be trained with images that contain various pieces of fruit, along with a *label* that specifies the class of fruit they represent (e.g. an apple, a banana, or a strawberry), and data specifying where each object appears in the image.

When an image is subsequently provided to the model, it will output a list of the objects it detects, the location of a bounding box that contains each object, and a score that indicates the confidence that detection was correct.

3.02 Module description

1. Input Signature

The model takes an image as input.

Let's assume the expected image is 300x300 pixels, with three channels (red, blue, and green) per pixel. This should be fed to the model as a flattened buffer of 270,000 byte values (300x300x3). If the model is quantized, each value should be a single byte representing a value between 0 and 255.

You can take a look at our example app code to understand how to do this pre-processing on Android.

2. Output Signature

The model outputs four arrays, mapped to the indices 0-4. Arrays 0, 1, and 2 describe N detected objects, with one element in each array corresponding to each object.

Index	Name	Description
0	Locations	Multidimensional array of [N][4] floating point values between 0 and 1, the inner arrays representing bounding boxes in the form [top, left, bottom, right]
1	Classes	Array of N integers (output as floating point values) each indicating the index of a class label from the labels file
2	Scores	Array of N floating point values between 0 and 1 representing probability that a class was detected
3	Number of detections	Integer value of N

The number of results is a parameter set while exporting the detection model to TensorFlow Lite.

For example, imagine a model has been trained to detect apples, bananas, and strawberries. When provided an image, it will output a set number of detection results - in this example;

Class	Score	Location
Apple	0.92	[18, 21, 57, 63]
Banana	0.88	[100, 30, 180, 150]
Strawberry	0.87	[7, 82, 89, 163]
Banana	0.23	[42, 66, 57, 83]
Apple	0.11	[6, 42, 31, 58]

1. Confidence score

To interpret these results, we can look at the score and the location for each detected object. The score is a number between 0 and 1 that indicates confidence that the object was genuinely detected. The closer the number is to 1, the more confident the model is.

Depending on your application, you can decide a cut-off threshold below which you will discard detection results. For the current example, a sensible cut-off is a score of 0.5 (meaning a 50% probability that the detection is valid). In that case, the last two objects in the array would be ignored because those confidence scores are below 0.5:

Class	Score	Location
Apple	0.92	[18, 21, 57, 63]
Banana	0.88	[100, 30, 180, 150]
Strawberry	0.87	[7, 82, 89, 163]
Banana	0.23	[42, 66, 57, 83]
Apple	0.11	[6, 42, 31, 58]

The cut-off you use should be based on whether you are more comfortable with false positives (objects that are wrongly identified, or areas of the image that are erroneously identified as objects when they are not), or false negatives (genuine objects that are missed because their confidence was low).

For example, in the following image, a pear (which is not an object that the model was trained to detect) was misidentified as a "person". This is an example of a false positive that could be ignored by selecting an appropriate cut-off. In this case, a cut-off of 0.6 (or 60%) would comfortably exclude the false positive.



2. Location

For each detected object, the model will return an array of four numbers representing a bounding rectangle that surrounds its position. For the starter model provided, the numbers are ordered as follows:

[top, left, bottom, right]

The top value represents the distance of the rectangle's top edge from the top of the image, in pixels. The left value represents the left edge's distance from the left of the input image. The other values represent the bottom and right edges in a similar manner.

Object detection models accept input images of a specific size. This is likely to be different from the size of the raw image captured by your device's camera, and you will have to write code to crop and scale your raw image to fit the model's input size (there are examples of this in our example applications).

The pixel values output by the model refer to the position in the cropped and scaled image, so you must scale them to fit the raw image in order to interpret them correctly.

3.03 Model Customization

Pre-trained models

Mobile-optimized detection models with a variety of latency and precision characteristics can be found in the Detection Zoo. Each one of them follows the input and output signatures described in the following sections.

Most of the download zips contain a model.tflite file. If there isn't one, a TensorFlow Lite flatbuffer can be generated using these instructions. SSD models from the TF2 Object Detection Zoo can also be converted to TensorFlow Lite using the instructions here. It is important to note that detection models cannot be converted directly using the TensorFlow Lite Converter, since they require an intermediate step of generating a mobile-friendly source model. The scripts linked above perform this step.

Both the TF1 & TF2 exporting scripts have parameters that can enable a larger number of output objects or slower, more-accurate post processing. Please use --help with the scripts to see an exhaustive list of supported arguments.

Currently, on-device inference is only optimized with SSD models. Better support for other architectures like CenterNet and EfficientDet is being investigated.

How to choose a model to customize?

Each model comes with its own precision (quantified by mAP value) and latency characteristics. You should choose a model that works the best for your use-case and intended hardware. For example, the Edge TPU models are ideal for inference on Google's Edge TPU on Pixel 4.

You can use our benchmark tool to evaluate models and choose the most efficient option available.

Fine-tuning models on custom data

The pre-trained models we provide are trained to detect 90 classes of objects. For a full list of classes, see the labels file in the model metadata.

You can use a technique known as transfer learning to re-train a model to recognize classes not in the original set. For example, you could re-train the model to detect multiple types of vegetable, despite there only being one vegetable in the original training data. To do this, you will need a set of training

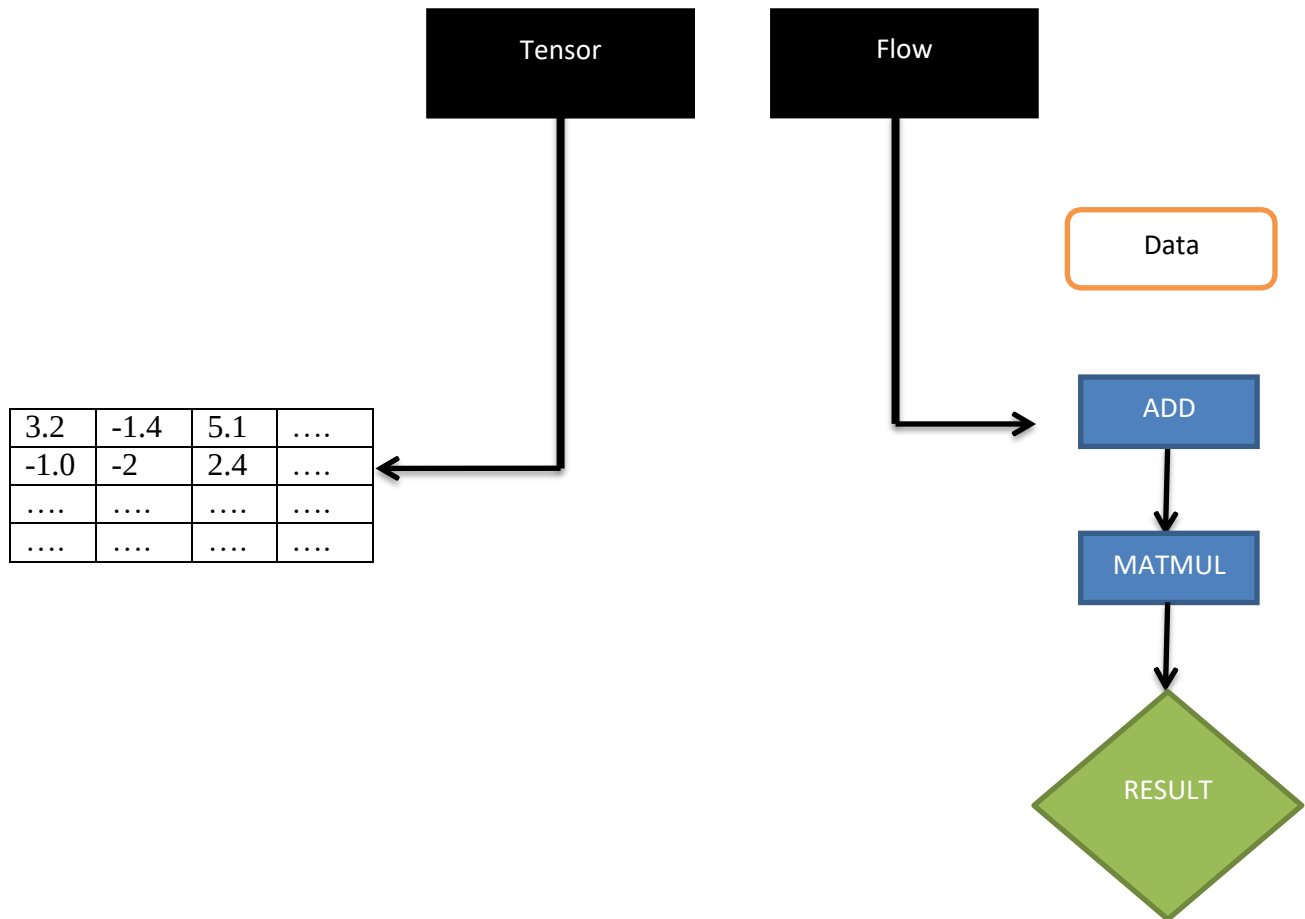
images for each of the new labels you wish to train. The recommended way is to use TensorFlow Lite Model Maker library which simplifies the process of training a TensorFlow Lite model using custom dataset, with a few lines of codes. It uses transfer learning to reduce the amount of required training data and time. You can also learn from Few-shot detection Colab as an example of fine-tuning a pre-trained model with few examples.

For fine-tuning with larger datasets, take a look at these guides for training your own models with the TensorFlow Object Detection API: TF1, TF2. Once trained, they can be converted to a TFLite-friendly format with the instructions here: TF1, TF2

3.11 Work Flow

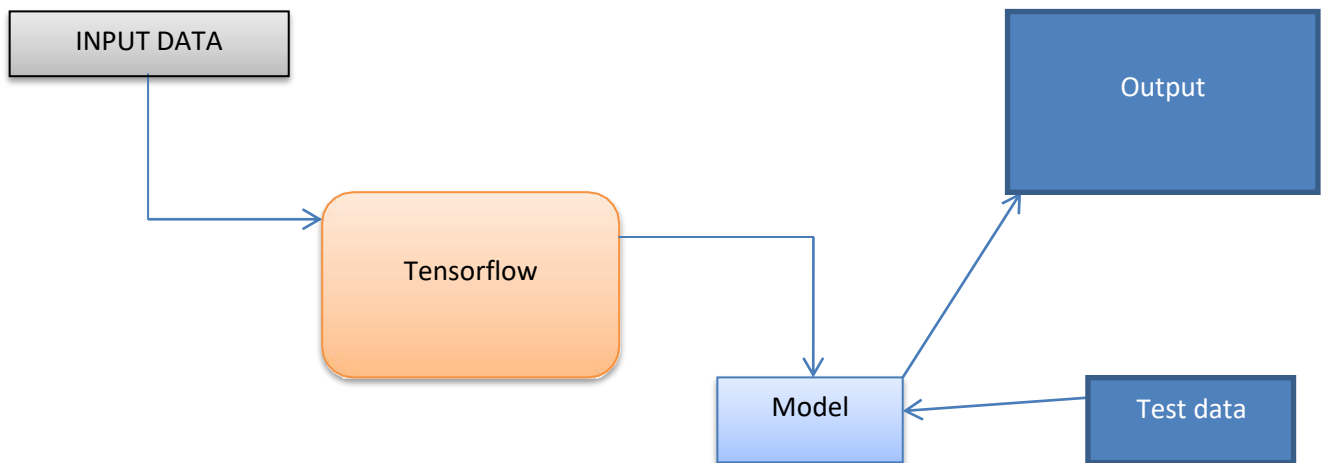
The Tensors are the standard way of representing data in deep learning. They are just multidimensional arrays, an extension of 2-dimensional tables(matrices) to data with higher dimension.

In Tensorflow, computation is approached as a dataflow graph. It can run on multiple CPUs and GPUs with optional CUDA and sy CL extensions for general-purpose computing on graphics processing unit and tensorflow is flexible architecture allows for easy deployment of computation across a variety of platforms which are the CPUs, GPUs and the TP use and from desktops to clusters to mobile and edge devices.



3.12 How Mobile Object Detection can be Performed in tensorflow

- We provide our input of data which is a set of images and using tensorflow, we train our model which is trained using deep learning and main objective of this model is extract the features which are based on edge detection ,HAR features, facial recognition.
- When these features are extracted, the model is then created to test our data which is again set a set of images in our case.
- We finally get an output in which we have the objects detected in an image



3.13 Dataset Sample View

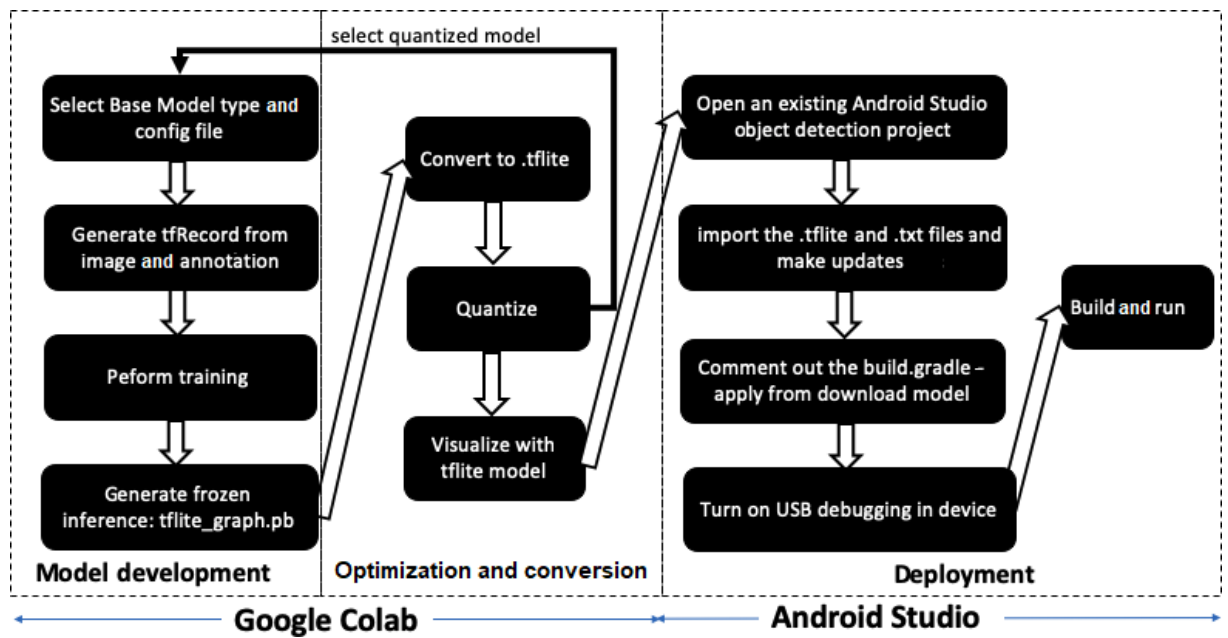
lite-model_ssd_mobilenet_v1_1_metadata_2.tflite									
1	2000	0000	5446	4c33	0000	0000	0000	1200	
2	1c00	1800	1400	1000	0c00	0800	0000	0400	
3	1200	0000	1800	0000	4400	0000	4056	3f00	
4	5056	3f00	90d8	3f00	0300	0000	0100	0000	
5	0c00	0000	0800	0c00	0800	0400	0800	0000	
6	b100	0000	0400	0000	0f00	0000	5446	4c49	
7	5445	5f4d	4554	4144	4154	4100	b200	0000	
8	f855	3f00	604d	3f00	444d	3f00	d44c	3f00	
9	3c44	3f00	cc43	3f00	343b	3f00	c43a	3f00	
10	2c32	3f00	9429	3f00	2429	3f00	d024	3f00	
11	9024	3f00	0023	3f00	3022	3f00	201f	3f00	
12	901d	3f00	001c	3f00	f015	3f00	e012	3f00	
13	d006	3f00	c0fa	3e00	b0f4	3e00	a0ee	3e00	
14	90e8	3e00	80e2	3e00	70dc	3e00	60d6	3e00	
15	50d0	3e00	40ca	3e00	30c7	3e00	20c4	3e00	
16	90c2	3e00	00c1	3e00	30c0	3e00	c0bf	3e00	
17	50bf	3e00	40b9	3e00	30b6	3e00	202e	3c00	
18	182e	3c00	08a6	3900	f815	3900	e8f5	2e00	
19	d853	2e00	c847	2e00	b82c	2e00	482c	2e00	
20	38ac	2900	28a6	2900	18a0	2900	0860	2700	
21	785e	2700	e85c	2700	d81c	2500	c8dc	2200	
22	38cf	2200	28af	2100	58a8	2100	4818	2100	
23	7811	2100	6811	1800	58c9	1700	e8c5	1700	
24	d8a1	1700	689e	1700	a89c	1700	1898	1700	
25	3097	1700	6096	1700	5896	1700	4856	1500	
26	4056	1500	b048	1500	a848	1500	9845	1500	
27	7845	1500	7045	1500	60df	0e00	58df	0e00	
28	48bb	0e00	40bb	0e00	38bb	0e00	30bb	0e00	
29	10bb	0e00	f4ba	0e00	d4ba	0e00	b8ba	0e00	
30	b0ba	0e00	94ba	0e00	909c	0e00	889c	0e00	
31	689c	0e00	609c	0e00	508a	0e00	488a	0e00	
32	2c8a	0e00	248a	0e00	947c	0e00	046f	0e00	
33	fc6e	0e00	f46e	0e00	ec6e	0e00	e46e	0e00	
34	d42e	0c00	b82e	0c00	b02e	0c00	942e	0c00	
35	8c2e	0c00	7c0a	0c00	6c04	0c00	6404	0c00	
36	5c04	0c00	5404	0c00	4c04	0c00	4404	0c00	
37	b4f6	0b00	acf6	0b00	9cae	0b00	94ae	0b00	
38	8cae	0b00	fca0	0b00	f4a0	0b00	eca0	0b00	
39	e4a0	0b00	dca0	0b00	d4a0	0b00	cca0	0b00	
40	bc97	0b00	b497	0b00	a485	0b00	9c85	0b00	
41	9485	0b00	7485	0b00	6c85	0b00	5c52	0800	
42	5452	0800	4c52	0800	4452	0800	3412	0600	
43	2c12	0600	2412	0600	1c12	0600	1412	0600	
44	0c12	0600	0412	0600	fc11	0600	f411	0600	
45	ec11	0600	dc0e	0600	d40e	0600	4475	0400	
46	3c75	0400	3475	0400	242d	0400	1c2d	0400	
47	142d	0400	0c2d	0400	042d	0400	fc2c	0400	
48	f42c	0400	ec2c	0400	5c93	0200	5493	0200	
49	1c00	0000	5446	4c33	0000	0000	0000	1200	

	lite-model_ssd_mobilenet_v1_1_metadata_2.tflite							
99	6573	2061	6620	7468	6520	6465	7465	6374
100	6564	2062	6f78	6573	2e00	0000	0800	0000
101	6361	7465	676f	7279	0000	0000	0c00	1200
102	0c00	0800	0000	0400	0c00	0000	1800	0000
103	5800	0000	8000	0000	0000	0a00	1000	0f00
104	0800	0400	0a00	0000	0c00	0000	1c00	0000
105	0000	0003	28ff	ffff	0200	0000	0200	0000
106	0800	0c00	0800	0700	0800	0000	0000	0001
107	0400	0000	0400	0000	0100	0000	0000	0000
108	0300	0000	0200	0000	2400	0000	5468	6520
109	6c6f	6361	7469	6f6e	7320	6f66	2074	6865
110	2064	6574	6563	7465	6420	626f	7865	732e
111	0000	0000	0800	0000	6c6f	6361	7469	6f6e
112	0000	0000	0100	0000	1400	0000	1000	1800
113	1400	1000	0000	0c00	0800	0400	1000	0000
114	1400	0000	2c00	0000	7000	0000	8400	0000
115	3401	0000	c8ff	ffff	0800	0000	0c00	0000
116	0100	0000	0000	0000	0100	0000	0000	7f43
117	0100	0000	0c00	0000	0800	0c00	0b00	0400
118	0800	0000	1000	0000	0000	0001	0800	0c00
119	0800	0400	0800	0000	0800	0000	0c00	0000
120	0100	0000	0000	ff42	0100	0000	0000	ff42
121	0800	0a00	0900	0400	0800	0000	0c00	0000
122	0002	0600	0800	0700	0600	0000	0000	0001
123	af00	0000	496e	7075	7420	696d	6167	6520
124	746f	2062	6520	6465	7465	6374	6564	2e20
125	5468	6520	6578	7065	6374	6564	2069	6d61
126	6765	2069	7320	3330	3020	7820	3330	302c
127	2077	6974	6820	7468	7265	6520	6368	616e
128	6e65	6c73	2028	7265	642c	2062	6c75	652c
129	2061	6e64	2067	7265	656e	2920	7065	7220
130	7069	7865	6c2e	2045	6163	6820	7661	6c75
131	6520	696e	2074	6865	2074	656e	736f	7220
132	6973	2061	2073	696e	676c	6520	6279	7465
133	2062	6574	7765	656e	2030	2061	6e64	2032
134	3535	2e00	0500	0000	696d	6167	6500	0000
135	0200	0000	7631	0000	9100	0000	4964	656e
136	7469	6679	2077	6869	6368	206f	6620	6120
137	6b6e	6f77	6e20	7365	7420	6f66	206f	626a
138	6563	7473	206d	6967	6874	2062	6520	7072
139	6573	656e	7420	616e	6420	7072	6f76	6964
140	6520	696e	666f	726d	6174	696f	6e20	6162
141	6f75	7420	7468	6569	7220	706f	7369	7469
142	6f6e	7320	7769	7468	696e	2074	6865	2067
143	6976	656e	2069	6d61	6765	206f	7220	6120
144	7669	6465	6f20	7374	7265	616d	2e00	0000
145	0f00	0000	5353	4420	4d6f	6269	6c65	4e65
146	7456	3100	e6a8	c0ff	0400	0000	8802	0000
147	999a	9395	908a	9898	9fa9	948b	h498	949c

	lite-model_ssd_mobilenet_v1_1_metadata_2.tflite							
201490	7970	7972	858c	7872	7278	7778	7577	7580
201491	7974	6e70	7379	6c67	8168	6873	8182	7873
201492	6a7f	7673	7c73	7775	7c66	7b78	7e73	717f
201493	6f7c	7b79	8671	8278	7872	7c73	6e74	7183
201494	7976	6277	766c	5f6c	6f7b	7375	755a	7d7b
201495	8165	7673	6e7e	7b70	8475	7975	6c6b	7673
201496	7c8e	7876	756f	7b64	6e79	7880	7777	7879
201497	776a	6b75	6e79	8071	6d76	6d7d	6473	7882
201498	7e75	7b67	7678	816d	837f	7174	7180	7d74
201499	7778	7b7d	6b8d	7a72	7a83	6d76	7833	7b71
201500	7881	7f6c	7271	7860	7283	7f74	7378	7078
201501	7977	7b68	6c67	7053	8a77	6076	7c69	7873
201502	7e71	7b5a	7376	787a	6d7b	7b6e	7b74	755d
201503	7a72	7772	7777	7a6e	7a5c	7575	7e73	7677
201504	7576	7b76	7a71	7578	7271	767e	7879	7b79
201505	7875	8479	8074	787d	7975	7b75	708a	727f
201506	7476	6f76	7077	7577	7876	6b71	6e77	7f7a
201507	7e70	746e	7b84	7964	8177	7177	6b74	737b
201508	7b77	7d76	7475	7263	867e	7773	7472	7a76
201509	6d71	727a	8367	7775	7673	7278	6477	7874
201510	817e	7b6c	6e83	8574	7a6a	8179	7584	7d72
201511	7c7c	767c	7675	7183	7383	846e	7970	767a
201512	7887	7778	766e	7175	7487	7476	8476	7579
201513	8074	726c	7179	7771	7778	6c7a	736e	7f79
201514	8378	7c85	9181	7d6e	7775	8f79	7f71	7589
201515	796c	7c72	7869	7261	7989	7a74	7b7d	7e72
201516	7371	777a	7d76	6b7c	7d75	7c78	757b	7b76
201517	6b78	8374	747d	7575	726f	8276	7471	7977
201518	7b7d	8182	776d	8277	8969	7c74	7780	906e
201519	8c6a	7e6c	7689	6f78	759f	806f	6f75	8875
201520	7e84	6775	7873	8472	6576	757c	637a	7c76
201521	7c7d	6c7a	7d6d	6e7a	7578	7d75	9974	7c87
201522	8482	7a6e	786d	746c	9592	7a77	8b81	7684
201523	6ba2	7176	747c	827e	987e	6d80	767a	7474
201524	887e	7181	707b	776b	7366	8285	796f	7370
201525	8289	7d66	7a7f	857f	6e85	6c7e	6e80	8280
201526	8471	6c77	9573	7e73	7675	916c	7e77	7396
201527	7e78	7b74	7b74	7b66	767d	8071	7980	7177
201528	7d72	7379	7b7b	717f	7f73	7471	7979	765e
201529	7174	827c	7876	767a	7274	8278	807f	7280
201530	7c7c	7786	7874	8f81	8073	7a6e	7b7b	956d
201531	8074	7971	7a88	7679	7785	7f6f	7274	8675
201532	7e79	8878	867b	8471	5d7f	7679	7073	7b6e
201533	8172	7377	7f6e	6c83	7a78	6e70	8c7d	7d8a
201534	8385	7e6c	727c	6b71	8596	7f79	8582	7585
201535	738e	7674	7086	7a7f	7878	6e71	7c7d	7677
201536	7d79	7282	7976	7363	7072	7b7e	7b6f	7c72
201537	807a	7d6d	7575	8978	667f	6f81	7271	7b80
201538	7878	8471	8873	7e77	7672	7a73	7f75	717f

lite-model_ssd_mobilenet_v1_1_metadata_2.tflite								
258888	c077	ffff	2a75	ffff	7477	ffff	a875	ffff
258889	c672	ffff	8c73	ffff	6773	ffff	a473	ffff
258890	5d71	ffff	3f72	ffff	a372	ffff	7e6f	ffff
258891	7172	ffff	2f77	ffff	866f	ffff	7f6f	ffff
258892	4772	ffff	2673	ffff	a974	ffff	9c70	ffff
258893	1f73	ffff	7271	ffff	6071	ffff	0b73	ffff
258894	f572	ffff	8170	ffff	2273	ffff	c374	ffff
258895	5972	ffff	9572	ffff	806f	ffff	e771	ffff
258896	4972	ffff	2872	ffff	1173	ffff	5172	ffff
258897	3375	ffff	fc74	ffff	c171	ffff	b974	ffff
258898	f471	ffff	9072	ffff	9772	ffff	ef71	ffff
258899	6e75	ffff	7073	ffff	5973	ffff	6578	ffff
258900	7a77	ffff	2e72	ffff	8a76	ffff	786f	ffff
258901	a07b	ffff	7a6f	ffff	776f	ffff	ec71	ffff
258902	846f	ffff	5874	ffff	2575	ffff	6f6f	ffff
258903	9e71	ffff	0273	ffff	3071	ffff	3b71	ffff
258904	4e73	ffff	2170	ffff	f673	ffff	6b71	ffff
258905	736f	ffff	0775	ffff	0370	ffff	c370	ffff
258906	8072	ffff	7973	ffff	c670	ffff	0772	ffff
258907	716f	ffff	32ad	ffff	7974	ffff	3475	ffff
258908	c875	ffff	f371	ffff	2774	ffff	f273	ffff
258909	0c75	ffff	b872	ffff	e771	ffff	ab72	ffff
258910	846f	ffff	3b70	ffff	7d71	ffff	f774	ffff
258911	4e76	ffff	4578	ffff	2578	ffff	4376	ffff
258912	1075	ffff	1475	ffff	7e76	ffff	9772	ffff
258913	9474	ffff	8678	ffff	776f	ffff	d873	ffff
258914	ee74	ffff	746f	ffff	736f	ffff	4e75	ffff
258915	0f76	ffff	ed74	ffff	2770	ffff	af72	ffff
258916	ac71	ffff	1a70	ffff	8c72	ffff	6173	ffff
258917	9770	ffff	f071	ffff	7d72	ffff	4673	ffff
258918	9474	ffff	796f	ffff	ef73	ffff	0b74	ffff
258919	6e72	ffff	8072	ffff	0b72	ffff	9472	ffff
258920	5e75	ffff	ac71	ffff	e873	ffff	a171	ffff
258921	0a73	ffff	5d72	ffff	9571	ffff	3f73	ffff
258922	ba72	ffff	7d73	ffff	b77b	ffff	5875	ffff
258923	2376	ffff	7a74	ffff	836f	ffff	4e76	ffff
258924	706f	ffff	7f6f	ffff	fd75	ffff	786f	ffff
258925	8d74	ffff	a375	ffff	9c6f	ffff	7571	ffff
258926	3e71	ffff	7572	ffff	8170	ffff	9b74	ffff
258927	0070	ffff	ad71	ffff	5a76	ffff	706f	ffff
258928	9373	ffff	fa70	ffff	cf73	ffff	dc72	ffff
258929	3b76	ffff	2971	ffff	2672	ffff	766f	ffff
258930	aa86	ffff	a071	ffff	0177	ffff	af72	ffff
258931	e176	ffff	7a72	ffff	d475	ffff	a474	ffff
258932	4576	ffff	ab70	ffff	1370	ffff	846f	ffff
258933	596f	ffff	2b70	ffff	2976	ffff	2f73	ffff
258934	1c74	ffff	9672	ffff	1d71	ffff	ff71	ffff
258935	b771	ffff	7471	ffff	2570	ffff	8c70	ffff

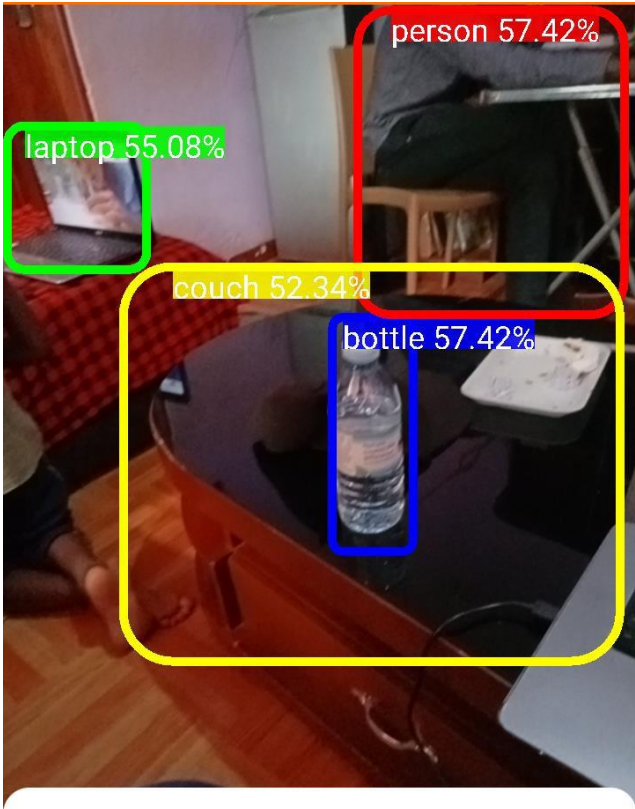
3.14 A detailed flowchat regarding object detection on Android Phones



CHAPTER 4

4.11 SYSTEM DEVELOPMENT

4.121 SCREENSHOTS



person 57.42%

laptop 55.08%

couch 52.34%

bottle 57.42%

Frame640x480

Crop300x300

Inference Time33ms

Threads

— 3 +

Developed By Kakooza Tobias

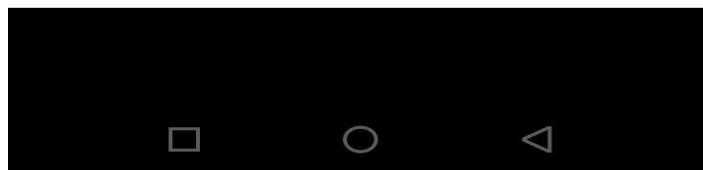


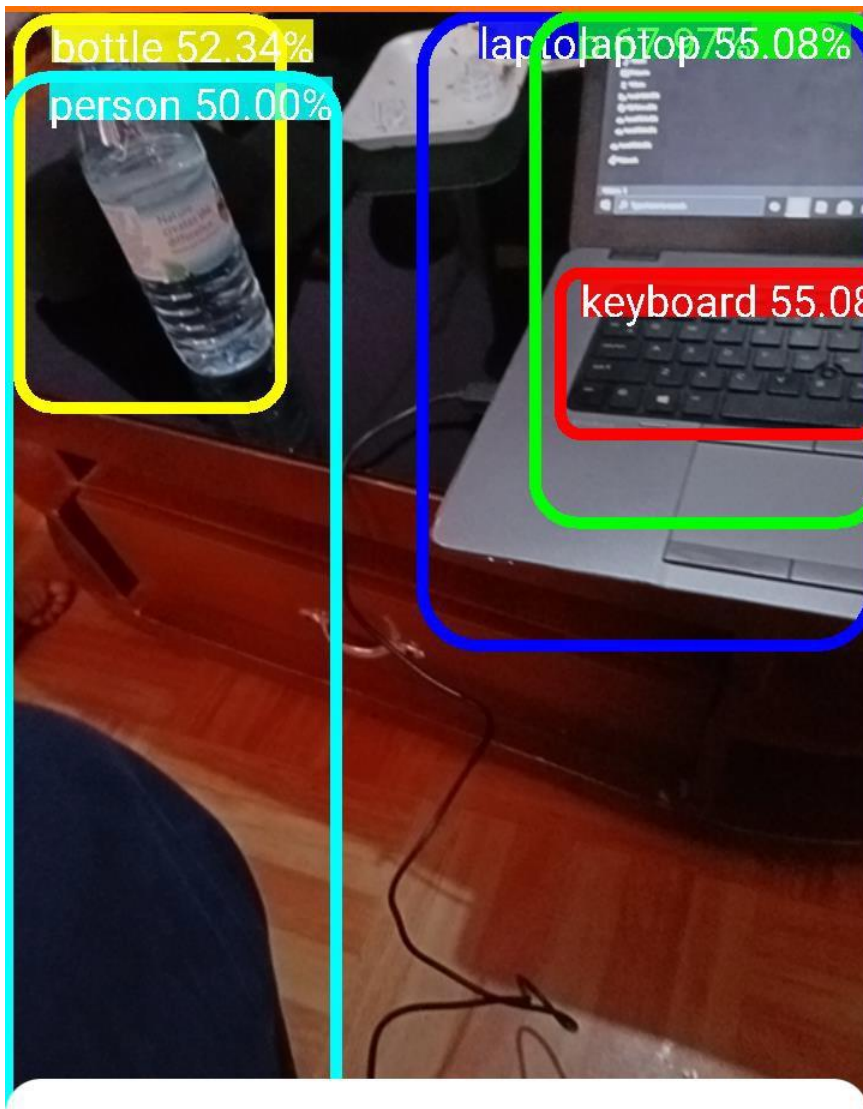
Frame	640x480
Crop	300x300
Inference Time	38ms

Threads

Developed By Kakooza Tobias

— 3 +





Frame	640x480
Crop	300x300
Inference Time	35ms

Threads

— 3 +

Developed By Kakooza Tobias





Frame 640x480

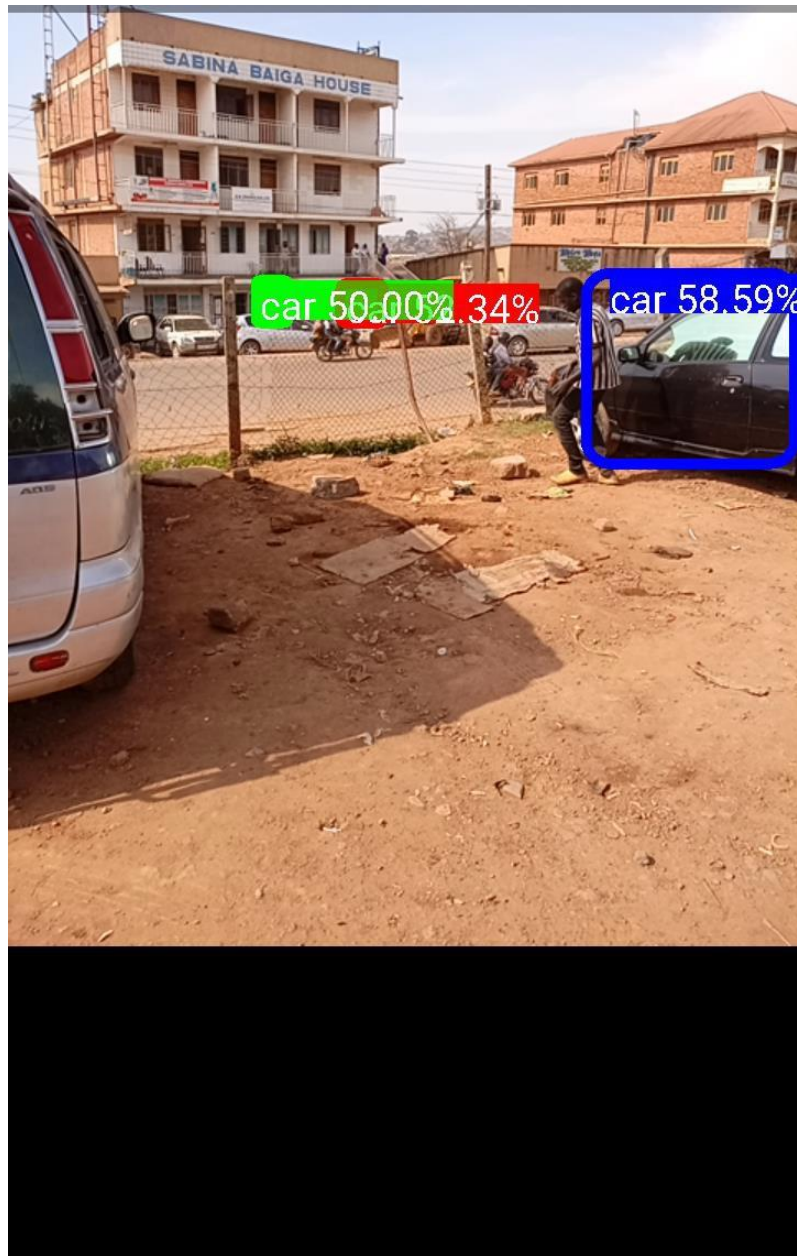
Crop 300x300

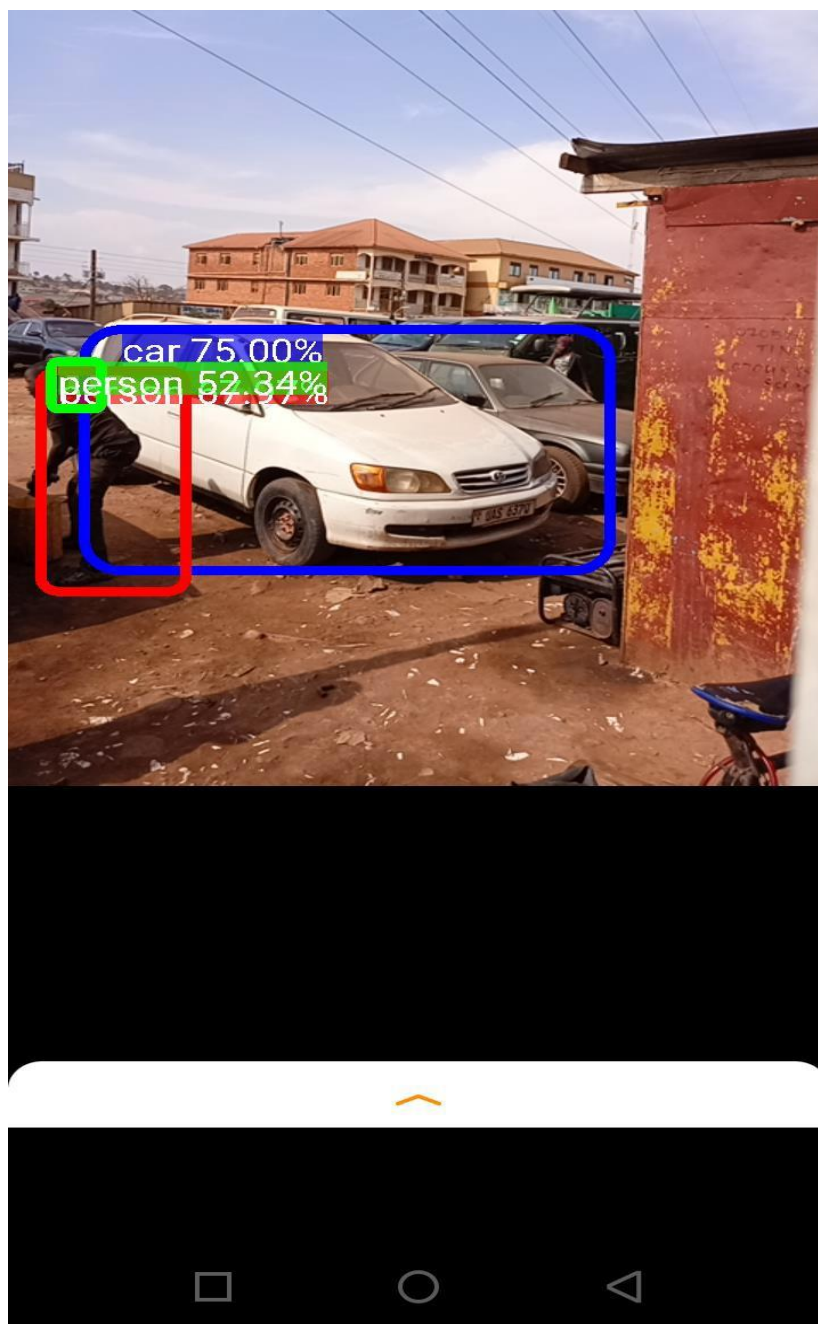
Inference Time 75ms

Threads

— 4 +

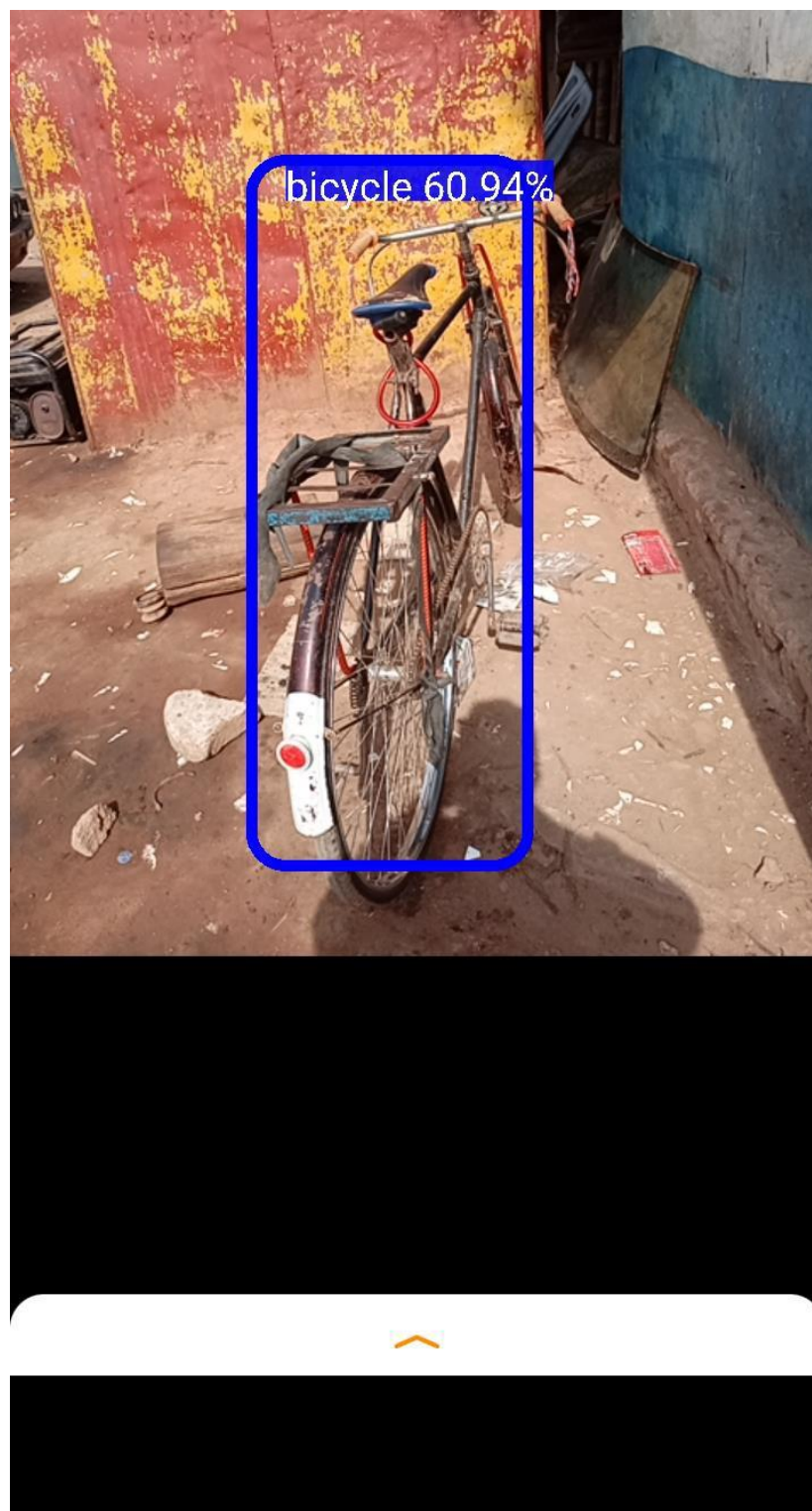














4.122 CODING

Layouts;

CameraActivity Xml Code

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#00000000">

    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="@android:color/black"
        android:orientation="vertical">

        <FrameLayout
            android:id="@+id/container"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            tools:context="org.tensorflow.demo.CameraActivity" >

            <androidx.appcompat.widget.Toolbar
                android:id="@+id/toolbar"
                android:layout_width="match_parent"
                android:layout_height="?attr/actionBarSize"
                android:background="@color/tfe_semi_transparent">

                <ImageView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:src="@drawable/detect"
                    android:textAlignment="center"
                />

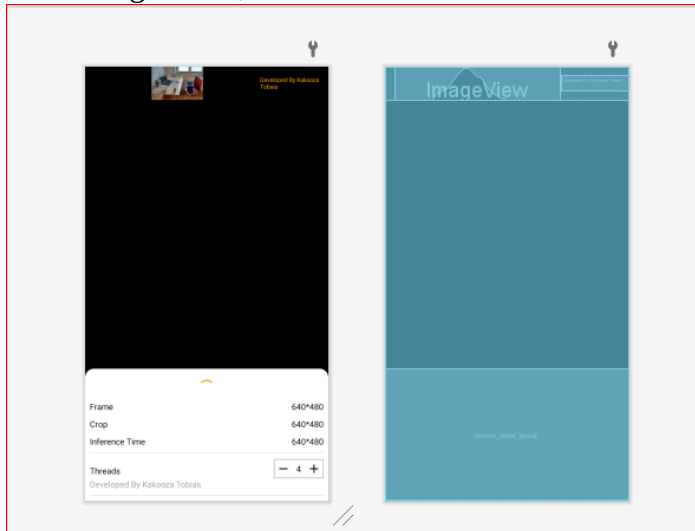
                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Developed By Kakooza Tobais"
                    android:textSize="10dp"
                    android:textColor="@color/tfe_color_primary"
                />

            </androidx.appcompat.widget.Toolbar>
        </FrameLayout>

    </RelativeLayout>

    <include
        android:id="@+id/bottom_sheet_layout"
        layout="@layout/tfe_od_layout_bottom_sheet" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Xml Design View;



CameraConnectionfragment Xml;

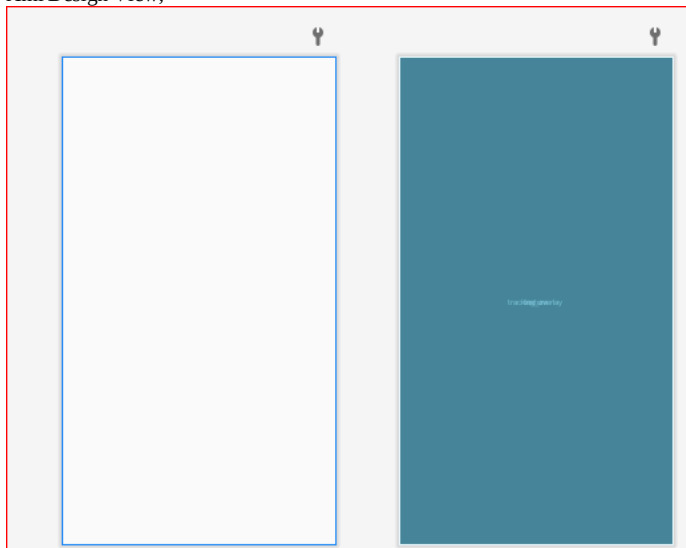
```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <org.tensorflow.lite.examples.detection.customview.AutoFitTextureView
        android:id="@+id/texture"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <org.tensorflow.lite.examples.detection.customview.OverlayView
        android:id="@+id/tracking_overlay"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</FrameLayout>
```

Xml Design View;



layoutBottomSheet Xml Code;

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/bottom_sheet_layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:background="@drawable/bottom_sheet_bg"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    android:padding="8dp"
    app:behavior_hideable="true"
    app:layout_behavior="com.google.android.material.bottomsheet.BottomSheetBehavior">

    <LinearLayout
        android:id="@+id/gesture_layout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingTop="10dp"
        android:paddingBottom="20dp">

        <ImageView
            android:id="@+id/bottom_sheet_arrow"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:src="@drawable/icn_chevron_up" />

    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/frame"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:text="Frame"
            android:textColor="@android:color/black" />

        <TextView
            android:id="@+id/frame_info"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:gravity="right"
            android:text="640*480"
            android:textColor="@android:color/black" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/crop"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:text="Crop"
            android:textColor="@android:color/black" />

        <TextView
```

```

        android:id="@+id/crop_info"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:gravity="right"
        android:text="640*480"
        android:textColor="@android:color/black" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/inference"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="Inference Time"
        android:textColor="@android:color/black" />

    <TextView
        android:id="@+id/inference_info"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:gravity="right"
        android:text="640*480"
        android:textColor="@android:color/black" />
</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1px"
    android:layout_marginTop="10dp"
    android:background="@android:color/darker_gray" />

<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:orientation="horizontal">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="Threads"
        android:textColor="@android:color/black" />

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"
        android:background="@drawable/rectangle"
        android:gravity="center"
        android:orientation="horizontal"
        android:padding="4dp">

        <ImageView
            android:id="@+id/minus"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:src="@drawable/ic_baseline_remove" />

        <TextView
            android:id="@+id/threads"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginLeft="10dp"

```

```

        android:layout_marginRight="10dp"
        android:text="4"
        android:textColor="@android:color/black"
        android:textSize="14sp" />

<ImageView
    android:id="@+id/plus"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/ic_baseline_add" />

</LinearLayout>

</RelativeLayout>
<TextView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:hint="Developed By Kakooza Tobias"/>

<View
    android:layout_width="match_parent"
    android:layout_height="1px"
    android:layout_marginTop="10dp"
    android:background="@android:color/darker_gray" />

<!-- TODO(b/112541284): Unhide when new NNAPI delegate is used. -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:visibility="gone">

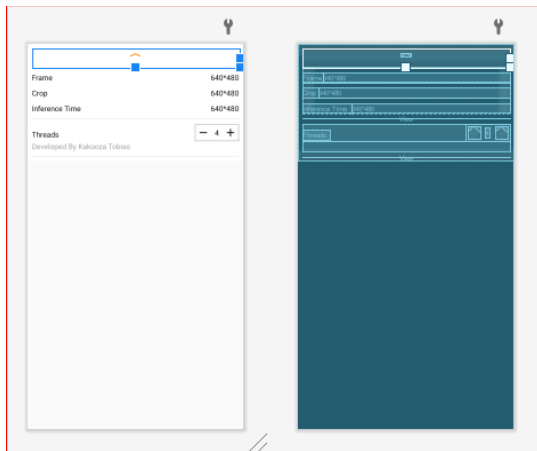
    <TextView
        android:id="@+id/api"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="API"
        android:textColor="@android:color/black" />

    <androidx.appcompat.widget.SwitchCompat
        android:id="@+id/api_info_switch"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:checked="false"
        android:gravity="right"
        android:text=""
        android:textColor="@android:color/black" />

</LinearLayout>
</LinearLayout>

```

layoutBottomSheet DesignView;



Values in Xml

colors

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <color name="tfe_color_primary">#ffa800</color>
  <color name="tfe_color_primary_dark">#ff6f00</color>
  <color name="tfe_color_accent">#425066</color>

  <color name="tfe_semi_transparent">#66000000</color>
</resources>
```

Dimens

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <dimen name="tfe_bottom_sheet_corner_radius">15dp</dimen>
  <dimen name="tfe_bottom_sheet_top_padding">8dp</dimen>
</resources>
```

Strings

```
<resources>
  <string name="tfe_od_app_name" translation_description="Object Detection ML app [CHAR_LIMIT=40]">Object Detect</string>
  <string name="tfe_od_camera_error" translation_description="Error regarding camera support[CHAR_LIMIT=40]">This device doesn't
support Camera2 API.</string>
</resources>
```

Styles

```
<resources>

  <!-- Base application theme. -->
  <style name="AppTheme.ObjectDetection" parent="Theme.AppCompat.Light.NoActionBar">
    <!-- Customize your theme here. -->
    <item name="colorPrimary">@color/tfe_color_primary</item>
    <item name="colorPrimaryDark">@color/tfe_color_primary_dark</item>
    <item name="colorAccent">@color/tfe_color_accent</item>
  </style>

</resources>
```

AndroidManifest in XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  package="org.tensorflow.lite.examples.detection">

  <uses-sdk />

  <uses-permission android:name="android.permission.CAMERA" />

  <uses-feature android:name="android.hardware.camera" />
  <uses-feature android:name="android.hardware.camera.autofocus" />
```

```

<application
    android:allowBackup="false"
    android:icon="@mipmap/obj"
    android:label="@string/tfe_od_app_name"
    android:roundIcon="@mipmap/obj"
    android:supportRtl="true"
    android:theme="@style/AppTheme.ObjectDetection">

    <activity
        android:name=".DetectorActivity"
        android:label="@string/tfe_od_app_name"
        android:screenOrientation="portrait">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

</application>
</manifest>

```

Class Files

CameraActivity

```

package org.tensorflow.lite.examples.detection;

import android.Manifest;

import android.app.Fragment;

import android.content.Context;

import android.content.pm.PackageManager;

import android.hardware.Camera;

import android.hardware.camera2.CameraAccessException;

import android.hardware.camera2.CameraCharacteristics;

import android.hardware.camera2.CameraManager;

import android.hardware.camera2.params.StreamConfigurationMap;

import android.media.Image;

import android.media.Image.Plane;

import android.media.ImageReader;

import android.media.ImageReader.OnImageAvailableListener;

import android.os.Build;

import android.os.Bundle;

import android.os.Handler;

import android.os.HandlerThread;

import android.os.Trace;

```

```

import androidx.appcompat.app.AppCompatActivity;

import androidx.appcompat.widget.SwitchCompat;

import androidx.appcompat.widget.Toolbar;

import android.util.Size;

import android.view.Surface;

import android.view.View;

import android.view.ViewTreeObserver;

import android.view.WindowManager;

import android.widget.CompoundButton;

import android.widget.ImageView;

import android.widget.LinearLayout;

import android.widget.TextView;

import android.widget.Toast;

import androidx.annotation.NonNull;

import com.google.android.material.bottomsheet.BottomSheetBehavior;

import java.nio.ByteBuffer;

import org.tensorflow.lite.examples.detection.env.ImageUtils;

import org.tensorflow.lite.examples.detection.env.Logger;


public abstract class CameraActivity extends AppCompatActivity

    implements OnImageAvailableListener,

        Camera.PreviewCallback,

        CompoundButton.OnCheckedChangeListener,

        View.OnClickListener {

    private static final Logger LOGGER = new Logger();


    private static final int PERMISSIONS_REQUEST = 1;


    private static final String PERMISSION_CAMERA = Manifest.permission.CAMERA;

    protected int previewWidth = 0;

    protected int previewHeight = 0;

    private boolean debug = false;

    private Handler handler;

```



```

private HandlerThread handlerThread;

private boolean useCamera2API;

private boolean isProcessingFrame = false;

private byte[][] yuvBytes = new byte[3][];

private int[] rgbBytes = null;

private int yRowStride;

private Runnable postInferenceCallback;

private Runnable imageConverter;


private LinearLayout bottomSheetLayout;

private LinearLayout gestureLayout;

private BottomSheetBehavior<LinearLayout> sheetBehavior;


protected TextView frameValueTextView, cropValueTextView, inferenceTimeTextView;

protected ImageView bottomSheetArrowImageView;

private ImageView plusImageView, minusImageView;

private SwitchCompat apiSwitchCompat;

private TextView threadsTextView;


@Override

protected void onCreate(final Bundle savedInstanceState) {

    LOGGER.d("onCreate " + this);

    super.onCreate(null);

    getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);


    setContentView(R.layout.tfe_od_activity_camera);

    Toolbar toolbar = findViewById(R.id.toolbar);

    setSupportActionBar(toolbar);

    getSupportActionBar().setDisplayShowTitleEnabled(false);


    if (hasPermission()) {

        setFragment();

    } else {

```

```

requestPermission();
}

threadsTextView = findViewById(R.id.threads);
plusImageView = findViewById(R.id.plus);
minusImageView = findViewById(R.id.minus);
apiSwitchCompat = findViewById(R.id.api_info_switch);
bottomSheetLayout = findViewById(R.id.bottom_sheet_layout);
gestureLayout = findViewById(R.id.gesture_layout);
sheetBehavior = BottomSheetBehavior.from(bottomSheetLayout);
bottomSheetArrowImageView = findViewById(R.id.bottom_sheet_arrow);

ViewTreeObserver vto = gestureLayout.getViewTreeObserver();
vto.addOnGlobalLayoutListener(
    new ViewTreeObserver.OnGlobalLayoutListener() {
        @Override
        public void onGlobalLayout() {
            if (Build.VERSION.SDK_INT < Build.VERSION_CODES.JELLY_BEAN) {
                gestureLayout.getViewTreeObserver().removeGlobalOnLayoutListener(this);
            } else {
                gestureLayout.getViewTreeObserver().removeOnGlobalLayoutListener(this);
            }
            // int width = bottomSheetLayout.getMeasuredWidth();
            int height = gestureLayout.getMeasuredHeight();

            sheetBehavior.setPeekHeight(height);
        }
    });
sheetBehavior.setHideable(false);

sheetBehavior.setBottomSheetCallback(
    new BottomSheetBehavior.BottomSheetCallback() {
        @Override

```

```

public void onStateChanged(@NonNull View bottomSheet, int newState) {

    switch (newState) {

        case BottomSheetBehavior.STATE_HIDDEN:

            break;

        case BottomSheetBehavior.STATE_EXPANDED:

            {

                bottomSheetArrowImageView.setImageResource(R.drawable.icn_chevron_down);

            }

            break;

        case BottomSheetBehavior.STATE_COLLAPSED:

            {

                bottomSheetArrowImageView.setImageResource(R.drawable.icn_chevron_up);

            }

            break;

        case BottomSheetBehavior.STATE_DRAGGING:

            break;

        case BottomSheetBehavior.STATE_SETTLING:

            bottomSheetArrowImageView.setImageResource(R.drawable.icn_chevron_up);

            break;

    }

}

@Override

public void onSlide(@NonNull View bottomSheet, float slideOffset) {}

});

frameValueTextView = findViewById(R.id.frame_info);

cropValueTextView = findViewById(R.id.crop_info);

inferenceTimeTextView = findViewById(R.id.inference_info);

apiSwitchCompat.setOnCheckedChangeListener(this);

plusImageView.setOnClickListener(this);

```

```

        minusImageView.setOnClickListener(this);
    }

    protected int[] getRgbBytes() {
        imageConverter.run();
        return rgbBytes;
    }

    protected int getLuminanceStride() {
        return yRowStride;
    }

    protected byte[] getLuminance() {
        return yuvBytes[0];
    }

    /** Callback for android.hardware.Camera API */
    @Override
    public void onPreviewFrame(final byte[] bytes, final Camera camera) {
        if (isProcessingFrame) {
            LOGGER.w("Dropping frame!");
            return;
        }

        try {
            // Initialize the storage bitmaps once when the resolution is known.
            if (rgbBytes == null) {
                Camera.Size previewSize = camera.getParameters().getPreviewSize();
                previewHeight = previewSize.height;
                previewWidth = previewSize.width;
                rgbBytes = new int[previewWidth * previewHeight];
                onPreviewSizeChosen(new Size(previewSize.width, previewSize.height), 90);
            }

```

```

    } catch (final Exception e) {

        LOGGER.e(e, "Exception!");

        return;
    }

```

```

isProcessingFrame = true;

yuvBytes[0] = bytes;

yRowStride = previewWidth;

```

```

imageConverter =

    new Runnable() {

        @Override

        public void run() {

            ImageUtils.convertYUV420SPToARGB8888(bytes, previewWidth, previewHeight, rgbBytes);

        }

    };

```

```

postInferenceCallback =

    new Runnable() {

        @Override

        public void run() {

            camera.addCallbackBuffer(bytes);

            isProcessingFrame = false;

        }

    };

processImage();
}

```

```

/** Callback for Camera2 API */

@Override

public void onImageAvailable(final ImageReader reader) {

    // We need wait until we have some size from onPreviewSizeChosen

    if (previewWidth == 0 || previewHeight == 0) {

```

```

return;

}

if (rgbBytes == null) {

    rgbBytes = new int[previewWidth * previewHeight];

}

try {

    final Image image = reader.acquireLatestImage();

    if (image == null) {

        return;

    }

    if (isProcessingFrame) {

        image.close();

        return;

    }

    isProcessingFrame = true;

    Trace.beginSection("imageAvailable");

    final Plane[] planes = image.getPlanes();

    fillBytes(planes, yuvBytes);

    yRowStride = planes[0].getRowStride();

    final int uvRowStride = planes[1].getRowStride();

    final int uvPixelStride = planes[1].getPixelStride();

    imageConverter =

        new Runnable() {

            @Override

            public void run() {

                ImageUtils.convertYUV420ToARGB8888(

                    yuvBytes[0],

                    yuvBytes[1],

                    yuvBytes[2],

                    previewWidth,

```

```

        previewHeight,
        yRowStride,
        uvRowStride,
        uvPixelStride,
        rgbBytes);
    }
};

```

```

postInferenceCallback =
    new Runnable() {
        @Override
        public void run() {
            image.close();
            isProcessingFrame = false;
        }
    };

```

```

        processImage();
    } catch (final Exception e) {
        LOGGER.e(e, "Exception!");
        Trace.endSection();
        return;
    }
    Trace.endSection();
}

```

```

@Override
public synchronized void onStart() {
    LOGGER.d("onStart " + this);
    super.onStart();
}

```

```

@Override

```

```

public synchronized void onResume() {

    LOGGER.d("onResume " + this);

    super.onResume();

    handlerThread = new HandlerThread("inference");

    handlerThread.start();

    handler = new Handler(handlerThread.getLooper());
}

```

@Override

```

public synchronized void onPause() {

    LOGGER.d("onPause " + this);

    handlerThread.quitSafely();

    try {

        handlerThread.join();

        handlerThread = null;

        handler = null;

    } catch (final InterruptedException e) {

        LOGGER.e(e, "Exception!");

    }

    super.onPause();
}

```

@Override

```

public synchronized void onStop() {

    LOGGER.d("onStop " + this);

    super.onStop();

}

```

@Override

```

public synchronized void onDestroy() {

```



```

    LOGGER.d("onDestroy " + this);

    super.onDestroy();
}

protected synchronized void runInBackground(final Runnable r) {

    if (handler != null) {

        handler.post(r);

    }

}

@Override

public void onRequestPermissionsResult(

    final int requestCode, final String[] permissions, final int[] grantResults) {

    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    if (requestCode == PERMISSIONS_REQUEST) {

        if (allPermissionsGranted(grantResults)) {

            setFragment();

        } else {

            requestPermission();

        }

    }

}

private static boolean allPermissionsGranted(final int[] grantResults) {

    for (int result : grantResults) {

        if (result != PackageManager.PERMISSION_GRANTED) {

            return false;

        }

    }

    return true;

}

private boolean hasPermission() {

```

```

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {

    return checkSelfPermission(PERMISSION_CAMERA) == PackageManager.PERMISSION_GRANTED;

} else {

    return true;

}

}

```

```

private void requestPermission() {

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {

        if (shouldShowRequestPermissionRationale(PERMISSION_CAMERA)) {

            Toast.makeText(

                CameraActivity.this,

                "Camera permission is required for this demo",

                Toast.LENGTH_LONG)

                .show();

        }

        requestPermissions(new String[] {PERMISSION_CAMERA}, PERMISSIONS_REQUEST);

    }

}

```

```

// Returns true if the device supports the required hardware level, or better.

private boolean isHardwareLevelSupported(

    CameraCharacteristics characteristics, int requiredLevel) {

    int deviceLevel = characteristics.get(CameraCharacteristics.INFO_SUPPORTED_HARDWARE_LEVEL);

    if (deviceLevel == CameraCharacteristics.INFO_SUPPORTED_HARDWARE_LEVEL_LEGACY) {

        return requiredLevel == deviceLevel;

    }

    // deviceLevel is not LEGACY, can use numerical sort

    return requiredLevel <= deviceLevel;

}

```

```

private String chooseCamera() {

    final CameraManager manager = (CameraManager) getSystemService(Context.CAMERA_SERVICE);

```

```

try {

    for (final String cameraId : manager.getCameraIdList()) {

        final CameraCharacteristics characteristics = manager.getCameraCharacteristics(cameraId);

        // We don't use a front facing camera in this sample.

        final Integer facing = characteristics.get(CameraCharacteristics.LENS_FACING);

        if (facing != null && facing == CameraCharacteristics.LENS_FACING_FRONT) {

            continue;

        }

        final StreamConfigurationMap map =

            characteristics.get(CameraCharacteristics.SCALER_STREAM_CONFIGURATION_MAP);

        if (map == null) {

            continue;

        }

        // Fallback to camera1 API for internal cameras that don't have full support.

        // This should help with legacy situations where using the camera2 API causes

        // distorted or otherwise broken previews.

        useCamera2API =

            (facing == CameraCharacteristics.LENS_FACING_EXTERNAL)

                || isHardwareLevelSupported(

                    characteristics, CameraCharacteristics.INFO_SUPPORTED_HARDWARE_LEVEL_FULL);

        LOGGER.i("Camera API lv2?: %s", useCamera2API);

        return cameraId;

    }

} catch (CameraAccessException e) {

    LOGGER.e(e, "Not allowed to access camera");

}

return null;

}

```

```

protected void setFragment() {

    String cameraId = chooseCamera();

    Fragment fragment;

    if (useCamera2API) {

        CameraConnectionFragment camera2Fragment =

            CameraConnectionFragment.newInstance(

                new CameraConnectionFragment.ConnectionCallback() {

                    @Override

                    public void onPreviewSizeChosen(final Size size, final int rotation) {

                        previewHeight = size.getHeight();

                        previewWidth = size.getWidth();

                        CameraActivity.this.onPreviewSizeChosen(size, rotation);

                    }

                },

                this,

                getLayoutId(),

                getDesiredPreviewFrameSize());

        camera2Fragment.setCamera(cameraId);

        fragment = camera2Fragment;

    } else {

        fragment =

            new LegacyCameraConnectionFragment(this, getLayoutId(), getDesiredPreviewFrameSize());

    }

    getFragmentManager().beginTransaction().replace(R.id.container, fragment).commit();

}

protected void fillBytes(final Plane[] planes, final byte[][] yuvBytes) {

    // Because of the variable row stride it's not possible to know in

    // advance the actual necessary dimensions of the yuv planes.

```

```

for (int i = 0; i < planes.length; ++i) {

    final ByteBuffer buffer = planes[i].getBuffer();

    if (yuvBytes[i] == null) {

        LOGGER.d("Initializing buffer %d at size %d", i, buffer.capacity());

        yuvBytes[i] = new byte[buffer.capacity()];

    }

    buffer.get(yuvBytes[i]);

}
}

```

```

public boolean isDebug() {

    return debug;

}

```

```

protected void readyForNextImage() {

    if (postInferenceCallback != null) {

        postInferenceCallback.run();

    }

}

```

```

protected int getScreenOrientation() {

    switch (getWindowManager().getDefaultDisplay().getRotation()) {

        case Surface.ROTATION_270:

            return 270;

        case Surface.ROTATION_180:

            return 180;

        case Surface.ROTATION_90:

            return 90;

        default:

            return 0;

    }

}

```

```

@Override

public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {

    setUseNNAPI(isChecked);

    if (isChecked) apiSwitchCompat.setText("NNAPI");

    else apiSwitchCompat.setText("TFLITE");

}

```

```

@Override

public void onClick(View v) {

    if (v.getId() == R.id.plus) {

        String threads = threadsTextView.getText().toString().trim();

        int numThreads = Integer.parseInt(threads);

        if (numThreads >= 9) return;

        numThreads++;

        threadsTextView.setText(String.valueOf(numThreads));

        setNumThreads(numThreads);

    } else if (v.getId() == R.id.minus) {

        String threads = threadsTextView.getText().toString().trim();

        int numThreads = Integer.parseInt(threads);

        if (numThreads == 1) {

            return;

        }

        numThreads--;

        threadsTextView.setText(String.valueOf(numThreads));

        setNumThreads(numThreads);

    }

}

```

```

protected void showFrameInfo(String frameInfo) {

    frameValueTextView.setText(frameInfo);

}

```

```

protected void showCropInfo(String cropInfo) {

```

```

cropValueTextView.setText(cropInfo);
}

protected void showInference(String inferenceTime) {
    inferenceTimeTextView.setText(inferenceTime);
}

protected abstract void processImage();

protected abstract void onPreviewSizeChosen(final Size size, final int rotation);

protected abstract int getLayoutId();

protected abstract Size getDesiredPreviewFrameSize();

protected abstract void setNumThreads(int numThreads);

protected abstract void setUseNNAPI(boolean isChecked);
}

```

CameraConnection

```

package org.tensorflow.lite.examples.detection;

import android.annotation.SuppressLint;
import android.app.Activity;
import android.app.AlertDialog;
import android.app.Dialog;
import android.app.DialogFragment;
import android.app.Fragment;
import android.content.Context;

```

```
import android.content.DialogInterface;

import android.content.res.Configuration;

import android.graphics.ImageFormat;

import android.graphics.Matrix;

import android.graphics.RectF;

import android.graphics.SurfaceTexture;

import android.hardware.camera2.CameraAccessException;

import android.hardware.camera2.CameraCaptureSession;

import android.hardware.camera2.CameraCharacteristics;

import android.hardware.camera2.CameraDevice;

import android.hardware.camera2.CameraManager;

import android.hardware.camera2.CaptureRequest;

import android.hardware.camera2.CaptureResult;

import android.hardware.camera2.TotalCaptureResult;

import android.hardware.camera2.params.StreamConfigurationMap;

import android.media.ImageReader;

import android.media.ImageReader.OnImageAvailableListener;

import android.os.Bundle;

import android.os.Handler;

import android.os.HandlerThread;

import android.text.TextUtils;

import android.util.Size;

import android.util.SparseIntArray;

import android.view.LayoutInflater;

import android.view.Surface;

import android.view.TextureView;

import android.view.View;

import android.view.ViewGroup;

import android.widget.Toast;
```



```

import java.util.ArrayList;

import java.util.Arrays;

import java.util.Collections;

import java.util.Comparator;

import java.util.List;

import java.util.concurrent.Semaphore;

import java.util.concurrent.TimeUnit;

import org.tensorflow.lite.examples.detection.customview.AutoFitTextureView;

import org.tensorflow.lite.examples.detection.env.Logger;


@SuppressLint("ValidFragment")

public class CameraConnectionFragment extends Fragment {

    private static final Logger LOGGER = new Logger();


    /**
     * The camera preview size will be chosen to be the smallest frame by pixel size capable of
     * containing a DESIRED_SIZE x DESIRED_SIZE square.
     */

    private static final int MINIMUM_PREVIEW_SIZE = 320;


    /** Conversion from screen rotation to JPEG orientation. */
    private static final SparseIntArray ORIENTATIONS = new SparseIntArray();


    private static final String FRAGMENT_DIALOG = "dialog";


    static {

        ORIENTATIONS.append(Surface.ROTATION_0, 90);

        ORIENTATIONS.append(Surface.ROTATION_90, 0);

        ORIENTATIONS.append(Surface.ROTATION_180, 270);
    }

```

```

    ORIENTATIONS.append(Surface.ROTATION_270, 180);
}

/** A {@link Semaphore} to prevent the app from exiting before closing the camera. */
private final Semaphore cameraOpenCloseLock = new Semaphore(1);

/** A {@link OnImageAvailableListener} to receive frames as they are available. */
private final OnImageAvailableListener imageListener;

/** The input size in pixels desired by TensorFlow (width and height of a square bitmap). */
private final Size inputSize;

/** The layout identifier to inflate for this Fragment. */
private final int layout;

private final ConnectionCallback cameraConnectionCallback;

private final CameraCaptureSession.CaptureCallback captureCallback =
    new CameraCaptureSession.CaptureCallback() {

        @Override
        public void onCaptureProgressed(
            final CameraCaptureSession session,
            final CaptureRequest request,
            final CaptureResult partialResult) {}

        @Override
        public void onCaptureCompleted(
            final CameraCaptureSession session,
            final CaptureRequest request,
            final TotalCaptureResult result) {}
    };

/** ID of the current {@link CameraDevice}. */
private String cameraId;

```

```

/** An {@link AutoFitTextureView} for camera preview. */
private AutoFitTextureView textureView;

/** A {@link CameraCaptureSession} for camera preview. */
private CameraCaptureSession captureSession;

/** A reference to the opened {@link CameraDevice}. */
private CameraDevice cameraDevice;

/** The rotation in degrees of the camera sensor from the display. */
private Integer sensorOrientation;

/** The {@link Size} of camera preview. */
private Size previewSize;

/** An additional thread for running tasks that shouldn't block the UI. */
private HandlerThread backgroundThread;

/** A {@link Handler} for running tasks in the background. */
private Handler backgroundHandler;

/** An {@link ImageReader} that handles preview frame capture. */
private ImageReader previewReader;

/** {@link CaptureRequest.Builder} for the camera preview */
private CaptureRequest.Builder previewRequestBuilder;

/** {@link CaptureRequest} generated by {@link #previewRequestBuilder} */
private CaptureRequest previewRequest;

/** {@link CameraDevice.StateCallback} is called when {@link CameraDevice} changes its state. */
private final CameraDevice.StateCallback stateCallback =
    new CameraDevice.StateCallback() {
        @Override
        public void onOpened(final CameraDevice cd) {
            // This method is called when the camera is opened. We start camera preview here.
            cameraOpenCloseLock.release();
            cameraDevice = cd;
            createCameraPreviewSession();
        }
    }

```

```
}
```

```
@Override
```

```
public void onDisconnected(final CameraDevice cd) {
```

```
    cameraOpenCloseLock.release();
```

```
    cd.close();
```

```
    cameraDevice = null;
```

```
}
```

```
@Override
```

```
public void onError(final CameraDevice cd, final int error) {
```

```
    cameraOpenCloseLock.release();
```

```
    cd.close();
```

```
    cameraDevice = null;
```

```
    final Activity activity = getActivity();
```

```
    if (null != activity) {
```

```
        activity.finish();
```

```
    }
```

```
}
```

```
};
```

```
/**
```

```
 * {@link TextureView.SurfaceTextureListener} handles several lifecycle events on a {@link
```

```
 * TextureView}.
```

```
 */
```

```
private final TextureView.SurfaceTextureListener surfaceTextureListener =
```

```
    new TextureView.SurfaceTextureListener() {
```

```
        @Override
```

```
        public void onSurfaceTextureAvailable(
```

```
            final SurfaceTexture texture, final int width, final int height) {
```

```

        openCamera(width, height);
    }

    @Override
    public void onSurfaceTextureSizeChanged(
        final SurfaceTexture texture, final int width, final int height) {
        configureTransform(width, height);
    }

    @Override
    public boolean onSurfaceTextureDestroyed(final SurfaceTexture texture) {
        return true;
    }

    @Override
    public void onSurfaceTextureUpdated(final SurfaceTexture texture) {}
};

private CameraConnectionFragment(
    final ConnectionCallback connectionCallback,
    final OnImageAvailableListener imageListener,
    final int layout,
    final Size inputSize) {
    this.cameraConnectionCallback = connectionCallback;
    this.imageListener = imageListener;
    this.layout = layout;
    this.inputSize = inputSize;
}

```

```

/**
 * Given {@code choices} of {@code Size}s supported by a camera, chooses the smallest one whose
 * width and height are at least as large as the minimum of both, or an exact match if possible.
 *
 * @param choices The list of sizes that the camera supports for the intended output class
 * @param width The minimum desired width
 * @param height The minimum desired height
 * @return The optimal {@code Size}, or an arbitrary one if none were big enough
 */
protected static Size chooseOptimalSize(final Size[] choices, final int width, final int height) {
    final int minSize = Math.max(Math.min(width, height), MINIMUM_PREVIEW_SIZE);
    final Size desiredSize = new Size(width, height);

    // Collect the supported resolutions that are at least as big as the preview Surface
    boolean exactSizeFound = false;

    final List<Size> bigEnough = new ArrayList<Size>();
    final List<Size> tooSmall = new ArrayList<Size>();
    for (final Size option : choices) {
        if (option.equals(desiredSize)) {
            // Set the size but don't return yet so that remaining sizes will still be logged.
            exactSizeFound = true;
        }

        if (option.getHeight() >= minSize && option.getWidth() >= minSize) {
            bigEnough.add(option);
        } else {
            tooSmall.add(option);
        }
    }
}

```

```

    LOGGER.i("Desired size: " + desiredSize + ", min size: " + minSize + "x" + minSize);

    LOGGER.i("Valid preview sizes: [" + TextUtils.join(", ", bigEnough) + "]");

    LOGGER.i("Rejected preview sizes: [" + TextUtils.join(", ", tooSmall) + "]");


    if (exactSizeFound) {

        LOGGER.i("Exact size match found.");

        return desiredSize;

    }


    // Pick the smallest of those, assuming we found any
    if (bigEnough.size() > 0) {

        final Size chosenSize = Collections.min(bigEnough, new CompareSizesByArea());

        LOGGER.i("Chosen size: " + chosenSize.getWidth() + "x" + chosenSize.getHeight());

        return chosenSize;

    } else {

        LOGGER.e("Couldn't find any suitable preview size");

        return choices[0];

    }

}


public static CameraConnectionFragment newInstance(

    final ConnectionCallback callback,

    final OnImageAvailableListener imageListener,

    final int layout,

    final Size inputSize) {

    return new CameraConnectionFragment(callback, imageListener, layout, inputSize);

}

```

```

/**
 * Shows a {@link Toast} on the UI thread.
 *
 * @param text The message to show
 */
private void showToast(final String text) {
    final Activity activity = getActivity();
    if (activity != null) {
        activity.runOnUiThread(
            new Runnable() {
                @Override
                public void run() {
                    Toast.makeText(activity, text, Toast.LENGTH_SHORT).show();
                }
            });
    }
}

@Override
public View onCreateView(
    final LayoutInflater inflater, final ViewGroup container, final Bundle savedInstanceState) {
    return inflater.inflate(layout, container, false);
}

@Override
public void onViewCreated(final View view, final Bundle savedInstanceState) {
    textureView = (AutoFitTextureView) view.findViewById(R.id.texture);
}

```



```
@Override

public void onActivityCreated(final Bundle savedInstanceState) {

    super.onActivityCreated(savedInstanceState);

}
```

```
@Override

public void onResume() {

    super.onResume();

    startBackgroundThread();


    // When the screen is turned off and turned back on, the SurfaceTexture is already
    // available, and "onSurfaceTextureAvailable" will not be called. In that case, we can open
    // a camera and start preview from here (otherwise, we wait until the surface is ready in
    // the SurfaceTextureListener).

    if (textureView.isAvailable()) {

        openCamera(textureView.getWidth(), textureView.getHeight());

    } else {

        textureView.setSurfaceTextureListener(surfaceTextureListener);

    }

}
```

```
@Override

public void onPause() {

    closeCamera();

    stopBackgroundThread();

    super.onPause();

}
```

```
public void setCamera(String cameraId) {
```

```

this.cameraId = cameraId;
}

/** Sets up member variables related to camera. */
private void setUpCameraOutputs() {
    final Activity activity = getActivity();
    final CameraManager manager = (CameraManager) activity.getSystemService(Context.CAMERA_SERVICE);
    try {
        final CameraCharacteristics characteristics = manager.getCameraCharacteristics(cameraId);

        final StreamConfigurationMap map =
            characteristics.get(CameraCharacteristics.SCALER_STREAM_CONFIGURATION_MAP);

        sensorOrientation = characteristics.get(CameraCharacteristics.SENSOR_ORIENTATION);

        // Danger, W.R.! Attempting to use too large a preview size could exceed the camera
        // bus' bandwidth limitation, resulting in gorgeous previews but the storage of
        // garbage capture data.
        previewSize =
            chooseOptimalSize(
                map.getOutputSizes(SurfaceTexture.class),
                inputSize.getWidth(),
                inputSize.getHeight());

        // We fit the aspect ratio of TextureView to the size of preview we picked.
        final int orientation = getResources().getConfiguration().orientation;
        if (orientation == Configuration.ORIENTATION_LANDSCAPE) {
            textureView.setAspectRatio(previewSize.getWidth(), previewSize.getHeight());
        } else {

```

```

        textureView.setAspectRatio(previewSize.getHeight(), previewSize.getWidth());
    }
} catch (final CameraAccessException e) {
    LOGGER.e(e, "Exception!");
} catch (final NullPointerException e) {
    // Currently an NPE is thrown when the Camera2API is used but not supported on the
    // device this code runs.

    AlertDialog.newInstance(getString(R.string.tfe_od_camera_error))
        .show(getChildFragmentManager(), FRAGMENT_DIALOG);

    throw new IllegalStateException(getString(R.string.tfe_od_camera_error));
}

cameraConnectionCallback.onPreviewSizeChosen(previewSize, sensorOrientation);
}

/** Opens the camera specified by {@link CameraConnectionFragment#cameraId}. */
private void openCamera(final int width, final int height) {
    setUpCameraOutputs();
    configureTransform(width, height);
    final Activity activity = getActivity();
    final CameraManager manager = (CameraManager) activity.getSystemService(Context.CAMERA_SERVICE);
    try {
        if (!cameraOpenCloseLock.tryAcquire(2500, TimeUnit.MILLISECONDS)) {
            throw new RuntimeException("Time out waiting to lock camera opening.");
        }

        manager.openCamera(cameraId, stateCallback, backgroundHandler);
    } catch (final CameraAccessException e) {
        LOGGER.e(e, "Exception!");
    } catch (final InterruptedException e) {

```

```

        throw new RuntimeException("Interrupted while trying to lock camera opening.", e);
    }
}

```

```

/** Closes the current {@link CameraDevice}. */
private void closeCamera() {
    try {
        cameraOpenCloseLock.acquire();

        if (null != captureSession) {
            captureSession.close();
            captureSession = null;
        }

        if (null != cameraDevice) {
            cameraDevice.close();
            cameraDevice = null;
        }

        if (null != previewReader) {
            previewReader.close();
            previewReader = null;
        }
    } catch (final InterruptedException e) {
        throw new RuntimeException("Interrupted while trying to lock camera closing.", e);
    } finally {
        cameraOpenCloseLock.release();
    }
}

```

```

/** Starts a background thread and its {@link Handler}. */
private void startBackgroundThread() {

```

```

backgroundThread = new HandlerThread("ImageListener");

backgroundThread.start();

backgroundHandler = new Handler(backgroundThread.getLooper());
}

/** Stops the background thread and its {@link Handler}. */
private void stopBackgroundThread() {

backgroundThread.quitSafely();

try {

    backgroundThread.join();

    backgroundThread = null;

    backgroundHandler = null;

} catch (final InterruptedException e) {

    LOGGER.e(e, "Exception!");

}

}

/** Creates a new {@link CameraCaptureSession} for camera preview. */
private void createCameraPreviewSession() {

try {

    final SurfaceTexture texture = textureView.getSurfaceTexture();

    assert texture != null;

    // We configure the size of default buffer to be the size of camera preview we want.

    texture.setDefaultBufferSize(previewSize.getWidth(), previewSize.getHeight());

    // This is the output Surface we need to start preview.

    final Surface surface = new Surface(texture);

    // We set up a CaptureRequest.Builder with the output Surface.

    previewRequestBuilder = cameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_PREVIEW);

    previewRequestBuilder.addTarget(surface);

```

```

LOGGER.i("Opening camera preview: " + previewSize.getWidth() + "x" + previewSize.getHeight());

// Create the reader for the preview frames.

previewReader =

    ImageReader.newInstance(

        previewSize.getWidth(), previewSize.getHeight(), ImageFormat.YUV_420_888, 2);

previewReader.setOnImageAvailableListener(imageListener, backgroundHandler);

previewRequestBuilder.addTarget(previewReader.getSurface());

// Here, we create a CameraCaptureSession for camera preview.

cameraDevice.createCaptureSession(

    Arrays.asList(surface, previewReader.getSurface()),

    new CameraCaptureSession.StateCallback() {

        @Override

        public void onConfigured(final CameraCaptureSession cameraCaptureSession) {

            // The camera is already closed

            if (null == cameraDevice) {

                return;

            }

            // When the session is ready, we start displaying the preview.

            captureSession = cameraCaptureSession;

            try {

                // Auto focus should be continuous for camera preview.

                previewRequestBuilder.set(

                    CaptureRequest.CONTROL_AF_MODE,

                    CaptureRequest.CONTROL_AF_MODE_CONTINUOUS_PICTURE);

                // Flash is automatically enabled when necessary.

                previewRequestBuilder.set(

                    CaptureRequest.CONTROL_AE_MODE, CaptureRequest.CONTROL_AE_MODE_ON_AUTO_FLASH);

```

```

        // Finally, we start displaying the camera preview.

        previewRequest = previewRequestBuilder.build();

        captureSession.setRepeatingRequest(

            previewRequest, captureCallback, backgroundHandler);

    } catch (final CameraAccessException e) {

        LOGGER.e(e, "Exception!");

    }

}

@Override

public void onConfigureFailed(final CameraCaptureSession cameraCaptureSession) {

    showToast("Failed");

}

},

null);

} catch (final CameraAccessException e) {

    LOGGER.e(e, "Exception!");

}

}

/**
 * Configures the necessary {@link Matrix} transformation to `mTextureView`. This method should be
 * called after the camera preview size is determined in setUpCameraOutputs and also the size of
 * `mTextureView` is fixed.
 *
 * @param viewWidth The width of `mTextureView`
 * @param viewHeight The height of `mTextureView`
 */
private void configureTransform(final int viewWidth, final int viewHeight) {

    final Activity activity = getActivity();

    if (null == textureView || null == previewSize || null == activity) {

        return;
    }

```

```

    }

    final int rotation = activity.getWindowManager().getDefaultDisplay().getRotation();

    final Matrix matrix = new Matrix();

    final RectF viewRect = new RectF(0, 0, viewWidth, viewHeight);

    final RectF bufferRect = new RectF(0, 0, previewSize.getHeight(), previewSize.getWidth());

    final float centerX = viewRect.centerX();

    final float centerY = viewRect.centerY();

    if (Surface.ROTATION_90 == rotation || Surface.ROTATION_270 == rotation) {

        bufferRect.offset(centerX - bufferRect.centerX(), centerY - bufferRect.centerY());

        matrix.setRectToRect(viewRect, bufferRect, Matrix.ScaleToFit.FILL);

        final float scale =

            Math.max(

                (float) viewHeight / previewSize.getHeight(),

                (float) viewWidth / previewSize.getWidth());

        matrix.postScale(scale, scale, centerX, centerY);

        matrix.postRotate(90 * (rotation - 2), centerX, centerY);

    } else if (Surface.ROTATION_180 == rotation) {

        matrix.postRotate(180, centerX, centerY);

    }

    textureView.setTransform(matrix);

}

/**

 * Callback for Activities to use to initialize their data once the selected preview size is

 * known.

 */

public interface ConnectionCallback {

    void onPreviewSizeChosen(Size size, int cameraRotation);

}

/** Compares two {@code Size}s based on their areas. */

static class CompareSizesByArea implements Comparator<Size> {

    @Override

```



```

public int compare(final Size lhs, final Size rhs) {

    // We cast here to ensure the multiplications won't overflow

    return Long.signum(

        (long) lhs.getWidth() * lhs.getHeight() - (long) rhs.getWidth() * rhs.getHeight());

    }

}

```

/** Shows an error message dialog. */

```

public static class AlertDialog extends DialogFragment {

    private static final String ARG_MESSAGE = "message";

    public static AlertDialog newInstance(final String message) {

        final AlertDialog dialog = new AlertDialog();

        final Bundle args = new Bundle();

        args.putString(ARG_MESSAGE, message);

        dialog.setArguments(args);

        return dialog;

    }

    @Override

    public Dialog onCreateDialog(final Bundle savedInstanceState) {

        final Activity activity = getActivity();

        return new AlertDialog.Builder(activity)

            .setMessage(getArguments().getString(ARG_MESSAGE))

            .setPositiveButton(

                android.R.string.ok,

                new DialogInterface.OnClickListener() {

                    @Override

                    public void onClick(final DialogInterface dialogInterface, final int i) {

                        activity.finish();

                    }

                })

            .create();

    }

}

```

```
}  
}  
}
```

DetectorActivity

```
package org.tensorflow.lite.examples.detection;  
  
import android.graphics.Bitmap;  
import android.graphics.Bitmap.Config;  
import android.graphics.Canvas;  
import android.graphics.Color;  
import android.graphics.Matrix;  
import android.graphics.Paint;  
import android.graphics.Paint.Style;  
import android.graphics.RectF;  
import android.graphics.Typeface;  
import android.media.ImageReader.OnImageAvailableListener;  
import android.os.SystemClock;  
import android.util.Size;  
import android.util.TypedValue;  
import android.widget.Toast;  
import java.io.IOException;  
import java.util.ArrayList;  
import java.util.List;  
import org.tensorflow.lite.examples.detection.customview.OverlayView;  
import org.tensorflow.lite.examples.detection.customview.OverlayView.DrawCallback;  
import org.tensorflow.lite.examples.detection.env.BorderedText;  
import org.tensorflow.lite.examples.detection.env.ImageUtils;  
import org.tensorflow.lite.examples.detection.env.Logger;  
import org.tensorflow.lite.examples.detection.tflite.Detector;  
import org.tensorflow.lite.examples.detection.tflite.TFLiteObjectDetectionAPIModel;  
import org.tensorflow.lite.examples.detection.tracking.MultiBoxTracker;
```

```

/**
 * An activity that uses a TensorFlowMultiBoxDetector and ObjectTracker to detect and then track
 * objects.
 */

public class DetectorActivity extends CameraActivity implements OnImageAvailableListener {

    private static final Logger LOGGER = new Logger();

    // Configuration values for the prepackaged SSD model.

    private static final int TF_OD_API_INPUT_SIZE = 300;

    private static final boolean TF_OD_API_IS_QUANTIZED = true;

    private static final String TF_OD_API_MODEL_FILE = "detect.tflite";

    private static final String TF_OD_API_LABELS_FILE = "labelmap.txt";

    private static final DetectorMode MODE = DetectorMode.TF_OD_API;

    // Minimum detection confidence to track a detection.

    private static final float MINIMUM_CONFIDENCE_TF_OD_API = 0.5f;

    private static final boolean MAINTAIN_ASPECT = false;

    private static final Size DESIRED_PREVIEW_SIZE = new Size(640, 480);

    private static final boolean SAVE_PREVIEW_BITMAP = false;

    private static final float TEXT_SIZE_DIP = 10;

    OverlayView trackingOverlay;

    private Integer sensorOrientation;

    private Detector detector;

    private long lastProcessingTimeMs;

    private Bitmap rgbFrameBitmap = null;

    private Bitmap croppedBitmap = null;

    private Bitmap cropCopyBitmap = null;

    private boolean computingDetection = false;

    private long timestamp = 0;

```

```

private Matrix frameToCropTransform;

private Matrix cropToFrameTransform;


private MultiBoxTracker tracker;


private BorderedText borderedText;


@Override

public void onPreviewSizeChosen(final Size size, final int rotation) {

    final float textSizePx =

        TypedValue.applyDimension(

            TypedValue.COMPLEX_UNIT_DIP, TEXT_SIZE_DIP, getResources().getDisplayMetrics());

    borderedText = new BorderedText(textSizePx);

    borderedText.setTypeface(Typeface.MONOSPACE);


    tracker = new MultiBoxTracker(this);


    int cropSize = TF_OD_API_INPUT_SIZE;


    try {

        detector =

            TFLiteObjectDetectionAPIModel.create(

                this,

                TF_OD_API_MODEL_FILE,

                TF_OD_API_LABELS_FILE,

                TF_OD_API_INPUT_SIZE,

                TF_OD_API_IS_QUANTIZED);

        cropSize = TF_OD_API_INPUT_SIZE;

    } catch (final IOException e) {

        e.printStackTrace();

        LOGGER.e(e, "Exception initializing Detector!");

        Toast toast =

```

```

        Toast.makeText(
            getApplicationContext(), "Detector could not be initialized", Toast.LENGTH_SHORT);
    toast.show();
    finish();
}

previewWidth = size.getWidth();
previewHeight = size.getHeight();

sensorOrientation = rotation - getScreenOrientation();
LOGGER.i("Camera orientation relative to screen canvas: %d", sensorOrientation);

LOGGER.i("Initializing at size %dx%d", previewWidth, previewHeight);
rgbFrameBitmap = Bitmap.createBitmap(previewWidth, previewHeight, Config.ARGB_8888);
croppedBitmap = Bitmap.createBitmap(cropSize, cropSize, Config.ARGB_8888);

frameToCropTransform =
    ImageUtils.getTransformationMatrix(
        previewWidth, previewHeight,
        cropSize, cropSize,
        sensorOrientation, MAINTAIN_ASPECT);

cropToFrameTransform = new Matrix();
frameToCropTransform.invert(cropToFrameTransform);

trackingOverlay = (OverlayView) findViewById(R.id.tracking_overlay);
trackingOverlay.addCallback(
    new DrawCallback() {
        @Override
        public void drawCallback(final Canvas canvas) {
            tracker.draw(canvas);
            if (isDebug()) {
                tracker.drawDebug(canvas);
            }
        }
    }
);

```

```

    }

    }

});

tracker.setFrameConfiguration(previewWidth, previewHeight, sensorOrientation);
}

@Override

protected void processImage() {

    ++timestamp;

    final long currTimestamp = timestamp;

    trackingOverlay.postInvalidate();

    // No mutex needed as this method is not reentrant.

    if (computingDetection) {

        readyForNextImage();

        return;

    }

    computingDetection = true;

    LOGGER.i("Preparing image " + currTimestamp + " for detection in bg thread.");

    rgbFrameBitmap.setPixels(getRgbBytes(), 0, previewWidth, 0, 0, previewWidth, previewHeight);

    readyForNextImage();

    final Canvas canvas = new Canvas(croppedBitmap);

    canvas.drawBitmap(rgbFrameBitmap, frameToCropTransform, null);

    // For examining the actual TF input.

    if (SAVE_PREVIEW_BITMAP) {

        ImageUtils.saveBitmap(croppedBitmap);

    }

    runInBackground(

```

```

new Runnable() {

    @Override

    public void run() {

        LOGGER.i("Running detection on image " + currTimestamp);

        final long startTime = SystemClock.uptimeMillis();

        final List<Detector.Recognition> results = detector.recognizeImage(croppedBitmap);

        lastProcessingTimeMs = SystemClock.uptimeMillis() - startTime;


        cropCopyBitmap = Bitmap.createBitmap(croppedBitmap);

        final Canvas canvas = new Canvas(cropCopyBitmap);

        final Paint paint = new Paint();

        paint.setColor(Color.RED);

        paint.setStyle(Style.STROKE);

        paint.setStrokeWidth(2.0f);


        float minimumConfidence = MINIMUM_CONFIDENCE_TF_OD_API;

        switch (MODE) {

            case TF_OD_API:

                minimumConfidence = MINIMUM_CONFIDENCE_TF_OD_API;

                break;

        }


        final List<Detector.Recognition> mappedRecognitions =

            new ArrayList<Detector.Recognition>();


        for (final Detector.Recognition result : results) {

            final RectF location = result.getLocation();

            if (location != null && result.getConfidence() >= minimumConfidence) {

                canvas.drawRect(location, paint);


                cropToFrameTransform.mapRect(location);


                result.setLocation(location);

```

```

        mappedRecognitions.add(result);
    }
}

tracker.trackResults(mappedRecognitions, currTimestamp);
trackingOverlay.postInvalidate();

computingDetection = false;

runOnUiThread(
    new Runnable() {
        @Override
        public void run() {
            showFrameInfo(previewWidth + "x" + previewHeight);
            showCropInfo(cropCopyBitmap.getWidth() + "x" + cropCopyBitmap.getHeight());
            showInference(lastProcessingTimeMs + "ms");
        }
    });
}

@Override
protected int getLayoutId() {
    return R.layout.tfe_od_camera_connection_fragment_tracking;
}

@Override
protected Size getDesiredPreviewFrameSize() {
    return DESIRED_PREVIEW_SIZE;
}

// Which detection model to use: by default uses Tensorflow Object Detection API frozen

```



```

// checkpoints.

private enum DetectorMode {

    TF_OD_API;

}

@Override

protected void setUseNNAPI(final boolean isChecked) {

    runInBackground(

        () -> {

            try {

                detector.setUseNNAPI(isChecked);

            } catch (UnsupportedOperationException e) {

                LOGGER.e(e, "Failed to set \"Use NNAPI\".");

                runOnUiThread(

                    () -> {

                        Toast.makeText(this, e.getMessage(), Toast.LENGTH_LONG).show();

                    });

            }

        });

}

```

```

@Override

protected void setNumThreads(final int numThreads) {

    runInBackground(() -> detector.setNumThreads(numThreads));

}

}

```

LegacyCameraConnectionFragment

```
package org.tensorflow.lite.examples.detection;
```

```

import android.app.Fragment;

import android.graphics.SurfaceTexture;

import android.hardware.Camera;

```

```

import android.hardware.Camera.CameraInfo;

import android.os.Bundle;

import android.os.Handler;

import android.os.HandlerThread;

import android.util.Size;

import android.util.SparseIntArray;

import android.view.LayoutInflater;

import android.view.Surface;

import android.view.TextureView;

import android.view.View;

import android.view.ViewGroup;

import java.io.IOException;

import java.util.List;

import org.tensorflow.lite.examples.detection.customview.AutoFitTextureView;

import org.tensorflow.lite.examples.detection.env.ImageUtils;

import org.tensorflow.lite.examples.detection.env.Logger;


public class LegacyCameraConnectionFragment extends Fragment {

    private static final Logger LOGGER = new Logger();

    /** Conversion from screen rotation to JPEG orientation. */

    private static final SparseIntArray ORIENTATIONS = new SparseIntArray();


    static {

        ORIENTATIONS.append(Surface.ROTATION_0, 90);

        ORIENTATIONS.append(Surface.ROTATION_90, 0);

        ORIENTATIONS.append(Surface.ROTATION_180, 270);

        ORIENTATIONS.append(Surface.ROTATION_270, 180);

    }


    private Camera camera;

    private Camera.PreviewCallback imageListener;

    private Size desiredSize;

    /** The layout identifier to inflate for this Fragment. */

```

```

private int layout;

/** An {@link AutoFitTextureView} for camera preview. */

private AutoFitTextureView textureView;

private SurfaceTexture availableSurfaceTexture = null;

/**
 * {@link TextureView.SurfaceTextureListener} handles several lifecycle events on a {@link
 * TextureView}.
 */

private final TextureView.SurfaceTextureListener surfaceTextureListener =
    new TextureView.SurfaceTextureListener() {

        @Override
        public void onSurfaceTextureAvailable(
            final SurfaceTexture texture, final int width, final int height) {

            availableSurfaceTexture = texture;

            startCamera();

        }

        @Override
        public void onSurfaceTextureSizeChanged(
            final SurfaceTexture texture, final int width, final int height) {}

        @Override
        public boolean onSurfaceTextureDestroyed(final SurfaceTexture texture) {

            return true;

        }

        @Override
        public void onSurfaceTextureUpdated(final SurfaceTexture texture) {}

    };

/** An additional thread for running tasks that shouldn't block the UI. */

private HandlerThread backgroundThread;

```

```

public LegacyCameraConnectionFragment(
    final Camera.PreviewCallback imageListener, final int layout, final Size desiredSize) {

    this.imageListener = imageListener;

    this.layout = layout;

    this.desiredSize = desiredSize;
}

@Override

public View onCreateView(
    final LayoutInflater inflater, final ViewGroup container, final Bundle savedInstanceState) {

    return inflater.inflate(layout, container, false);
}

@Override

public void onViewCreated(final View view, final Bundle savedInstanceState) {

    textureView = (AutoFitTextureView) view.findViewById(R.id.texture);
}

@Override

public void onActivityCreated(final Bundle savedInstanceState) {

    super.onActivityCreated(savedInstanceState);
}

@Override

public void onResume() {

    super.onResume();

    startBackgroundThread();

    // When the screen is turned off and turned back on, the SurfaceTexture is already
    // available, and "onSurfaceTextureAvailable" will not be called. In that case, we can open
    // a camera and start preview from here (otherwise, we wait until the surface is ready in
    // the SurfaceTextureListener).

    if (textureView.isAvailable()) {

```

```

        startCamera();

    } else {

        textureView.setSurfaceTextureListener(surfaceTextureListener);

    }

}

```

```

@Override

public void onPause() {

    stopCamera();

    stopBackgroundThread();

    super.onPause();

}

```

```

/** Starts a background thread and its {@link Handler}. */

private void startBackgroundThread() {

    backgroundThread = new HandlerThread("CameraBackground");

    backgroundThread.start();

}

```

```

/** Stops the background thread and its {@link Handler}. */

private void stopBackgroundThread() {

    backgroundThread.quitSafely();

    try {

        backgroundThread.join();

        backgroundThread = null;

    } catch (final InterruptedException e) {

        LOGGER.e(e, "Exception!");

    }

}

```

```

private void startCamera() {

    int index = getCameraId();

    camera = Camera.open(index);

```

```

try {

    Camera.Parameters parameters = camera.getParameters();

    List<String> focusModes = parameters.getSupportedFocusModes();

    if (focusModes != null

        && focusModes.contains(Camera.Parameters.FOCUS_MODE_CONTINUOUS_PICTURE)) {

        parameters.setFocusMode(Camera.Parameters.FOCUS_MODE_CONTINUOUS_PICTURE);

    }

    List<Camera.Size> cameraSizes = parameters.getSupportedPreviewSizes();

    Size[] sizes = new Size[cameraSizes.size()];

    int i = 0;

    for (Camera.Size size : cameraSizes) {

        sizes[i++] = new Size(size.width, size.height);

    }

    Size previewSize =

        CameraConnectionFragment.chooseOptimalSize(

            sizes, desiredSize.getWidth(), desiredSize.getHeight());

    parameters.setPreviewSize(previewSize.getWidth(), previewSize.getHeight());

    camera.setDisplayOrientation(90);

    camera.setParameters(parameters);

    camera.setPreviewTexture(availableSurfaceTexture);

} catch (IOException exception) {

    camera.release();

}

camera.setPreviewCallbackWithBuffer(imageListener);

Camera.Size s = camera.getParameters().getPreviewSize();

camera.addCallbackBuffer(new byte[ImageUtils.getYUVByteSize(s.height, s.width)]);

textureView.setAspectRatio(s.height, s.width);

camera.startPreview();

}

```

```

protected void stopCamera() {

    if (camera != null) {

        camera.stopPreview();

        camera.setPreviewCallback(null);

        camera.release();

        camera = null;

    }

}

private int getCameraId() {

    CameraInfo ci = new CameraInfo();

    for (int i = 0; i < Camera.getNumberOfCameras(); i++) {

        Camera.getCameraInfo(i, ci);

        if (ci.facing == CameraInfo.CAMERA_FACING_BACK) return i;

    }

    return -1; // No camera found

}

}

```

CustomView classes

AutoFit TextureView

```

package org.tensorflow.lite.examples.detection.customview;

import android.content.Context;

import android.util.AttributeSet;

import android.view.TextureView;

/** A {@link TextureView} that can be adjusted to a specified aspect ratio. */
public class AutoFitTextureView extends TextureView {

    private int ratioWidth = 0;

    private int ratioHeight = 0;

```

```

public AutoFitTextureView(final Context context) {

    this(context, null);

}

public AutoFitTextureView(final Context context, final AttributeSet attrs) {

    this(context, attrs, 0);

}

public AutoFitTextureView(final Context context, final AttributeSet attrs, final int defStyle) {

    super(context, attrs, defStyle);

}

/**
 * Sets the aspect ratio for this view. The size of the view will be measured based on the ratio
 * calculated from the parameters. Note that the actual sizes of parameters don't matter, that is,
 * calling setAspectRatio(2, 3) and setAspectRatio(4, 6) make the same result.
 *
 * @param width Relative horizontal size
 * @param height Relative vertical size
 */
public void setAspectRatio(final int width, final int height) {

    if (width < 0 || height < 0) {

        throw new IllegalArgumentException("Size cannot be negative.");

    }

    ratioWidth = width;

    ratioHeight = height;

    requestLayout();

}

@Override

protected void onMeasure(final int widthMeasureSpec, final int heightMeasureSpec) {

    super.onMeasure(widthMeasureSpec, heightMeasureSpec);

    final int width = MeasureSpec.getSize(widthMeasureSpec);

```



```

final int height = MeasureSpec.getSize(heightMeasureSpec);

if (0 == ratioWidth || 0 == ratioHeight) {

    setMeasuredDimension(width, height);

} else {

    if (width < height * ratioWidth / ratioHeight) {

        setMeasuredDimension(width, width * ratioHeight / ratioWidth);

    } else {

        setMeasuredDimension(height * ratioWidth / ratioHeight, height);

    }

}

}

}

```

OverlayView

```

package org.tensorflow.lite.examples.detection.customview;

import android.content.Context;

import android.graphics.Canvas;

import android.util.AttributeSet;

import android.view.View;

import java.util.LinkedList;

import java.util.List;

/** A simple View providing a render callback to other classes. */

public class OverlayView extends View {

    private final List<DrawCallback> callbacks = new LinkedList<DrawCallback>();

    public OverlayView(final Context context, final AttributeSet attrs) {

        super(context, attrs);

    }

    public void addCallback(final DrawCallback callback) {

```

```

callbacks.add(callback);
}

@Override

public synchronized void draw(final Canvas canvas) {

    for (final DrawCallback callback : callbacks) {

        callback.drawCallback(canvas);

    }

}

/** Interface defining the callback for client classes. */
public interface DrawCallback {

    public void drawCallback(final Canvas canvas);

}
}

```

RecognitionScoreView

```

package org.tensorflow.lite.examples.detection.customview;

import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Paint;
import android.util.AttributeSet;
import android.util.TypedValue;
import android.view.View;
import java.util.List;
import org.tensorflow.lite.examples.detection.tflite.Detector.Recognition;

public class RecognitionScoreView extends View implements ResultsView {

    private static final float TEXT_SIZE_DIP = 14;

    private final float textSizePx;

    private final Paint fgPaint;

    private final Paint bgPaint;

```

```

private List<Recognition> results;

public RecognitionScoreView(final Context context, final AttributeSet set) {
    super(context, set);

    textSizePx =
        TypedValue.applyDimension(
            TypedValue.COMPLEX_UNIT_DIP, TEXT_SIZE_DIP, getResources().getDisplayMetrics());
    fgPaint = new Paint();
    fgPaint.setTextSize(textSizePx);

    bgPaint = new Paint();
    bgPaint.setColor(0xcc4285f4);
}

@Override
public void setResults(final List<Recognition> results) {
    this.results = results;
    postInvalidate();
}

@Override
public void onDraw(final Canvas canvas) {
    final int x = 10;
    int y = (int) (fgPaint.getTextSize() * 1.5f);

    canvas.drawPaint(bgPaint);

    if (results != null) {
        for (final Recognition recog : results) {
            canvas.drawText(recog.getTitle() + ": " + recog.getConfidence(), x, y, fgPaint);
            y += (int) (fgPaint.getTextSize() * 1.5f);
        }
    }
}

```

```
}  
  
}  
  
}
```

ResultsView

```
package org.tensorflow.lite.examples.detection.customview;  
  
import java.util.List;  
  
import org.tensorflow.lite.examples.detection.tflite.Detector.Recognition;  
  
public interface ResultsView {  
  
    public void setResults(final List<Recognition> results);  
  
}
```

Env Classes

BorderedText

```
package org.tensorflow.lite.examples.detection.env;  
  
import android.graphics.Canvas;  
  
import android.graphics.Color;  
  
import android.graphics.Paint;  
  
import android.graphics.Paint.Align;  
  
import android.graphics.Paint.Style;  
  
import android.graphics.Rect;  
  
import android.graphics.Typeface;  
  
import java.util.Vector;  
  
/** A class that encapsulates the tedious bits of rendering legible, bordered text onto a canvas. */  
public class BorderedText {  
  
    private final Paint interiorPaint;  
  
    private final Paint exteriorPaint;
```

```

private final float textSize;

/**
 * Creates a left-aligned bordered text object with a white interior, and a black exterior with
 * the specified text size.
 *
 * @param textSize text size in pixels
 */
public BorderedText(final float textSize) {
    this(Color.WHITE, Color.BLACK, textSize);
}

/**
 * Create a bordered text object with the specified interior and exterior colors, text size and
 * alignment.
 *
 * @param interiorColor the interior text color
 * @param exteriorColor the exterior text color
 * @param textSize text size in pixels
 */
public BorderedText(final int interiorColor, final int exteriorColor, final float textSize) {
    interiorPaint = new Paint();
    interiorPaint.setTextSize(textSize);
    interiorPaint.setColor(interiorColor);
    interiorPaint.setStyle(Style.FILL);
    interiorPaint.setAntiAlias(false);
    interiorPaint.setAlpha(255);

    exteriorPaint = new Paint();
    exteriorPaint.setTextSize(textSize);
    exteriorPaint.setColor(exteriorColor);
    exteriorPaint.setStyle(Style.FILL_AND_STROKE);
    exteriorPaint.setStrokeWidth(textSize / 8);
}

```

```
exteriorPaint.setAntiAlias(false);
```

```
exteriorPaint.setAlpha(255);
```

```
this.textSize = textSize;
```

```
}
```

```
public void setTypeface(Typeface typeface) {
```

```
    interiorPaint.setTypeface(typeface);
```

```
    exteriorPaint.setTypeface(typeface);
```

```
}
```

```
public void drawText(final Canvas canvas, final float posX, final float posY, final String text) {
```

```
    canvas.drawText(text, posX, posY, exteriorPaint);
```

```
    canvas.drawText(text, posX, posY, interiorPaint);
```

```
}
```

```
public void drawText(
```

```
    final Canvas canvas, final float posX, final float posY, final String text, Paint bgPaint) {
```

```
    float width = exteriorPaint.measureText(text);
```

```
    float textSize = exteriorPaint.getTextSize();
```

```
    Paint paint = new Paint(bgPaint);
```

```
    paint.setStyle(Paint.Style.FILL);
```

```
    paint.setAlpha(160);
```

```
    canvas.drawRect(posX, (posY + (int) (textSize)), (posX + (int) (width)), posY, paint);
```

```
    canvas.drawText(text, posX, (posY + textSize), interiorPaint);
```

```
}
```

```
public void drawLines(Canvas canvas, final float posX, final float posY, Vector<String> lines) {
```

```
    int lineNum = 0;
```

```
    for (final String line : lines) {
```

```
        drawText(canvas, posX, posY - getTextSize() * (lines.size() - lineNum - 1), line);
```

```

        ++lineNum;
    }
}

public void setInteriorColor(final int color) {
    interiorPaint.setColor(color);
}

public void setExteriorColor(final int color) {
    exteriorPaint.setColor(color);
}

public float getTextSize() {
    return textSize;
}

public void setAlpha(final int alpha) {
    interiorPaint.setAlpha(alpha);
    exteriorPaint.setAlpha(alpha);
}

public void getTextBounds(
    final String line, final int index, final int count, final Rect lineBounds) {
    interiorPaint.getTextBounds(line, index, count, lineBounds);
}

public void setTextAlign(final Align align) {
    interiorPaint.setTextAlign(align);
    exteriorPaint.setTextAlign(align);
}
}

```

ImageUtils

```

package org.tensorflow.lite.examples.detection.env;

import android.graphics.Bitmap;

import android.graphics.Matrix;

import android.os.Environment;

import java.io.File;

import java.io.FileOutputStream;

/** Utility class for manipulating images. */

public class ImageUtils {

    // This value is 2 ^ 18 - 1, and is used to clamp the RGB values before their ranges
    // are normalized to eight bits.

    static final int kMaxChannelValue = 262143;

    @SuppressWarnings("unused")

    private static final Logger LOGGER = new Logger();

    /**
     * Utility method to compute the allocated size in bytes of a YUV420SP image of the given
     * dimensions.
     */

    public static int getYUVByteSize(final int width, final int height) {

        // The luminance plane requires 1 byte per pixel.

        final int ySize = width * height;

        // The UV plane works on 2x2 blocks, so dimensions with odd size must be rounded up.

        // Each 2x2 block takes 2 bytes to encode, one each for U and V.

        final int uvSize = ((width + 1) / 2) * ((height + 1) / 2) * 2;

        return ySize + uvSize;

    }

```



```

/**
 * Saves a Bitmap object to disk for analysis.
 *
 * @param bitmap The bitmap to save.
 */
public static void saveBitmap(final Bitmap bitmap) {
    saveBitmap(bitmap, "preview.png");
}

/**
 * Saves a Bitmap object to disk for analysis.
 *
 * @param bitmap The bitmap to save.
 * @param filename The location to save the bitmap to.
 */
public static void saveBitmap(final Bitmap bitmap, final String filename) {
    final String root =
        Environment.getExternalStorageDirectory().getAbsolutePath() + File.separator + "tensorflow";
    LOGGER.i("Saving %dx%d bitmap to %s.", bitmap.getWidth(), bitmap.getHeight(), root);
    final File myDir = new File(root);

    if (!myDir.mkdirs()) {
        LOGGER.i("Make dir failed");
    }

    final String fname = filename;
    final File file = new File(myDir, fname);

    if (file.exists()) {
        file.delete();
    }

    try {
        final FileOutputStream out = new FileOutputStream(file);
        bitmap.compress(Bitmap.CompressFormat.PNG, 99, out);
    }
}

```

```

        out.flush();

        out.close();
    } catch (final Exception e) {

        LOGGER.e(e, "Exception!");
    }
}

public static void convertYUV420SPToARGB8888(byte[] input, int width, int height, int[] output) {

    final int frameSize = width * height;

    for (int j = 0, yp = 0; j < height; j++) {

        int uvp = frameSize + (j >> 1) * width;

        int u = 0;

        int v = 0;

        for (int i = 0; i < width; i++, yp++) {

            int y = 0xff & input[yp];

            if ((i & 1) == 0) {

                v = 0xff & input[uvp++];

                u = 0xff & input[uvp++];
            }

            output[yp] = YUV2RGB(y, u, v);
        }
    }
}

private static int YUV2RGB(int y, int u, int v) {

    // Adjust and check YUV values

    y = (y - 16) < 0 ? 0 : (y - 16);

    u -= 128;

    v -= 128;

    // This is the floating point equivalent. We do the conversion in integer

```

```

// because some Android devices do not have floating point in hardware.

// nR = (int)(1.164 * nY + 2.018 * nU);

// nG = (int)(1.164 * nY - 0.813 * nV - 0.391 * nU);

// nB = (int)(1.164 * nY + 1.596 * nV);

int y1192 = 1192 * y;

int r = (y1192 + 1634 * v);

int g = (y1192 - 833 * v - 400 * u);

int b = (y1192 + 2066 * u);

// Clipping RGB values to be inside boundaries [ 0 , kMaxChannelValue ]

r = r > kMaxChannelValue ? kMaxChannelValue : (r < 0 ? 0 : r);

g = g > kMaxChannelValue ? kMaxChannelValue : (g < 0 ? 0 : g);

b = b > kMaxChannelValue ? kMaxChannelValue : (b < 0 ? 0 : b);

return 0xff000000 | ((r << 6) & 0xff0000) | ((g >> 2) & 0xff00) | ((b >> 10) & 0xff);
}

```

```

public static void convertYUV420ToARGB8888(

```

```

    byte[] yData,

    byte[] uData,

    byte[] vData,

    int width,

    int height,

    int yRowStride,

    int uvRowStride,

    int uvPixelStride,

    int[] out) {

    int yp = 0;

    for (int j = 0; j < height; j++) {

        int pY = yRowStride * j;

        int pUV = uvRowStride * (j >> 1);

        for (int i = 0; i < width; i++) {

```

```

int uv_offset = pUV + (i >> 1) * uvPixelStride;

out[yp++] = YUV2RGB(0xff & yData[pY + i], 0xff & uData[uv_offset], 0xff & vData[uv_offset]);
}
}
}

/**
 * Returns a transformation matrix from one reference frame into another. Handles cropping (if
 * maintaining aspect ratio is desired) and rotation.
 *
 * @param srcWidth Width of source frame.
 * @param srcHeight Height of source frame.
 * @param dstWidth Width of destination frame.
 * @param dstHeight Height of destination frame.
 * @param applyRotation Amount of rotation to apply from one frame to another. Must be a multiple
 * of 90.
 * @param maintainAspectRatio If true, will ensure that scaling in x and y remains constant,
 * cropping the image if necessary.
 * @return The transformation fulfilling the desired requirements.
 */
public static Matrix getTransformationMatrix(
    final int srcWidth,
    final int srcHeight,
    final int dstWidth,
    final int dstHeight,
    final int applyRotation,
    final boolean maintainAspectRatio) {
    final Matrix matrix = new Matrix();

    if (applyRotation != 0) {
        if (applyRotation % 90 != 0) {
            LOGGER.w("Rotation of %d % 90 != 0", applyRotation);

```

```

}

// Translate so center of image is at origin.
matrix.postTranslate(-srcWidth / 2.0f, -srcHeight / 2.0f);

// Rotate around origin.
matrix.postRotate(applyRotation);
}

// Account for the already applied rotation, if any, and then determine how
// much scaling is needed for each axis.
final boolean transpose = (Math.abs(applyRotation) + 90) % 180 == 0;

final int inWidth = transpose ? srcHeight : srcWidth;
final int inHeight = transpose ? srcWidth : srcHeight;

// Apply scaling if necessary.
if (inWidth != dstWidth || inHeight != dstHeight) {
    final float scaleFactorX = dstWidth / (float) inWidth;
    final float scaleFactorY = dstHeight / (float) inHeight;

    if (maintainAspectRatio) {
        // Scale by minimum factor so that dst is filled completely while
        // maintaining the aspect ratio. Some image may fall off the edge.
        final float scaleFactor = Math.max(scaleFactorX, scaleFactorY);
        matrix.postScale(scaleFactor, scaleFactor);
    } else {
        // Scale exactly to fill dst from src.
        matrix.postScale(scaleFactorX, scaleFactorY);
    }
}

if (applyRotation != 0) {

```

```

        // Translate back from origin centered reference to destination frame.
        matrix.postTranslate(dstWidth / 2.0f, dstHeight / 2.0f);
    }

    return matrix;
}
}

```

Logger

```

package org.tensorflow.lite.examples.detection.env;

import android.util.Log;
import java.util.HashSet;
import java.util.Set;

/** Wrapper for the platform log function, allows convenient message prefixing and log disabling. */
public final class Logger {

    private static final String DEFAULT_TAG = "tensorflow";

    private static final int DEFAULT_MIN_LOG_LEVEL = Log.DEBUG;

    // Classes to be ignored when examining the stack trace
    private static final Set<String> IGNORED_CLASS_NAMES;

    static {

        IGNORED_CLASS_NAMES = new HashSet<String>(3);

        IGNORED_CLASS_NAMES.add("dalvik.system.VMStack");

        IGNORED_CLASS_NAMES.add("java.lang.Thread");

        IGNORED_CLASS_NAMES.add(Logger.class.getCanonicalName());

    }

    private final String tag;

```

```

private final String messagePrefix;

private int minLogLevel = DEFAULT_MIN_LOG_LEVEL;

/**
 * Creates a Logger using the class name as the message prefix.
 *
 * @param clazz the simple name of this class is used as the message prefix.
 */
public Logger(final Class<?> clazz) {
    this(clazz.getSimpleName());
}

/**
 * Creates a Logger using the specified message prefix.
 *
 * @param messagePrefix is prepended to the text of every message.
 */
public Logger(final String messagePrefix) {
    this(DEFAULT_TAG, messagePrefix);
}

/**
 * Creates a Logger with a custom tag and a custom message prefix. If the message prefix is set to
 *
 * <pre>null</pre>
 *
 * , the caller's class name is used as the prefix.
 *
 * @param tag identifies the source of a log message.
 * @param messagePrefix prepended to every message if non-null. If null, the name of the caller is
 * being used
 */
public Logger(final String tag, final String messagePrefix) {

```

```

this.tag = tag;

final String prefix = messagePrefix == null ? getCallerSimpleName() : messagePrefix;

this.messagePrefix = (prefix.length() > 0) ? prefix + ": " : prefix;
}

/** Creates a Logger using the caller's class name as the message prefix. */

public Logger() {

    this(DEFAULT_TAG, null);

}

/** Creates a Logger using the caller's class name as the message prefix. */

public Logger(final int minLogLevel) {

    this(DEFAULT_TAG, null);

    this.minLogLevel = minLogLevel;

}

/**

    * Return caller's simple name.

    *

    * <p>Android getStackTrace() returns an array that looks like this: stackTrace[0]:

    * dalvik.system.VMStack stackTrace[1]: java.lang.Thread stackTrace[2]:

    * com.google.android.apps.unveil.env.UnveilLogger stackTrace[3]:

    * com.google.android.apps.unveil.BaseApplication

    *

    * <p>This function returns the simple version of the first non-filtered name.

    *

    * @return caller's simple name

    */

private static String getCallerSimpleName() {

    // Get the current callstack so we can pull the class of the caller off of it.

    final StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();

    for (final StackTraceElement elem : stackTrace) {

```



```

final String className = elem.getClassName();

if (!IGNORED_CLASS_NAMES.contains(className)) {

    // We're only interested in the simple name of the class, not the complete package.

    final String[] classParts = className.split("\\.");

    return classParts[classParts.length - 1];

}

}

```

```

return Logger.class.getSimpleName();

}

```

```

public void setMinLogLevel(final int minLogLevel) {

    this.minLogLevel = minLogLevel;

}

```

```

public boolean isLoggable(final int logLevel) {

    return logLevel >= minLogLevel || Log.isLoggable(tag, logLevel);

}

```

```

private String toMessage(final String format, final Object... args) {

    return messagePrefix + (args.length > 0 ? String.format(format, args) : format);

}

```

```

public void v(final String format, final Object... args) {

    if (isLoggable(Log.VERBOSE)) {

        Log.v(tag, toMessage(format, args));

    }

}

```

```

public void v(final Throwable t, final String format, final Object... args) {

    if (isLoggable(Log.VERBOSE)) {

        Log.v(tag, toMessage(format, args), t);

    }

}

```

```
}
```

```
public void d(final String format, final Object... args) {
```

```
    if (isLoggable(Log.DEBUG)) {
```

```
        Log.d(tag, toMessage(format, args));
```

```
    }
```

```
}
```

```
public void d(final Throwable t, final String format, final Object... args) {
```

```
    if (isLoggable(Log.DEBUG)) {
```

```
        Log.d(tag, toMessage(format, args), t);
```

```
    }
```

```
}
```

```
public void i(final String format, final Object... args) {
```

```
    if (isLoggable(Log.INFO)) {
```

```
        Log.i(tag, toMessage(format, args));
```

```
    }
```

```
}
```

```
public void i(final Throwable t, final String format, final Object... args) {
```

```
    if (isLoggable(Log.INFO)) {
```

```
        Log.i(tag, toMessage(format, args), t);
```

```
    }
```

```
}
```

```
public void w(final String format, final Object... args) {
```

```
    if (isLoggable(Log.WARN)) {
```

```
        Log.w(tag, toMessage(format, args));
```

```
    }
```

```
}
```

```
public void w(final Throwable t, final String format, final Object... args) {
```

```

    if (isLoggable(Log.WARN)) {
        Log.w(tag, toMessage(format, args), t);
    }
}

public void e(final String format, final Object... args) {
    if (isLoggable(Log.ERROR)) {
        Log.e(tag, toMessage(format, args));
    }
}

public void e(final Throwable t, final String format, final Object... args) {
    if (isLoggable(Log.ERROR)) {
        Log.e(tag, toMessage(format, args), t);
    }
}
}

```

Size

```

package org.tensorflow.lite.examples.detection.env;

import android.graphics.Bitmap;
import android.text.TextUtils;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

/** Size class independent of a Camera object. */
public class Size implements Comparable<Size>, Serializable {

    // 1.4 went out with this UID so we'll need to maintain it to preserve pending queries when
    // upgrading.

    public static final long serialVersionUID = 7689808733290872361L;

```

```

public final int width;

public final int height;

public Size(final int width, final int height) {

    this.width = width;

    this.height = height;

}

public Size(final Bitmap bmp) {

    this.width = bmp.getWidth();

    this.height = bmp.getHeight();

}

/**
 * Rotate a size by the given number of degrees.
 *
 * @param size Size to rotate.
 * @param rotation Degrees {0, 90, 180, 270} to rotate the size.
 * @return Rotated size.
 */
public static Size getRotatedSize(final Size size, final int rotation) {

    if (rotation % 180 != 0) {

        // The phone is portrait, therefore the camera is sideways and frame should be rotated.

        return new Size(size.height, size.width);

    }

    return size;

}

public static Size parseFromString(String sizeString) {

    if (TextUtils.isEmpty(sizeString)) {

        return null;

    }

```

```
sizeString = sizeString.trim();
```

```
// The expected format is "<width>x<height>".
```

```
final String[] components = sizeString.split("x");
```

```
if (components.length == 2) {
```

```
    try {
```

```
        final int width = Integer.parseInt(components[0]);
```

```
        final int height = Integer.parseInt(components[1]);
```

```
        return new Size(width, height);
```

```
    } catch (final NumberFormatException e) {
```

```
        return null;
```

```
    }
```

```
} else {
```

```
    return null;
```

```
}
```

```
}
```

```
public static List<Size> sizeStringToList(final String sizes) {
```

```
    final List<Size> sizeList = new ArrayList<Size>();
```

```
    if (sizes != null) {
```

```
        final String[] pairs = sizes.split(",");
```

```
        for (final String pair : pairs) {
```

```
            final Size size = Size.parseFromString(pair);
```

```
            if (size != null) {
```

```
                sizeList.add(size);
```

```
            }
```

```
        }
```

```
    }
```

```
    return sizeList;
```

```
}
```

```
public static String sizeListToString(final List<Size> sizes) {
```

```

String sizesString = "";

if (sizes != null && sizes.size() > 0) {

    sizesString = sizes.get(0).toString();

    for (int i = 1; i < sizes.size(); i++) {

        sizesString += "," + sizes.get(i).toString();

    }

}

return sizesString;

}

public static final String dimensionsAsString(final int width, final int height) {

    return width + "x" + height;

}

public final float aspectRatio() {

    return (float) width / (float) height;

}

@Override

public int compareTo(final Size other) {

    return width * height - other.width * other.height;

}

@Override

public boolean equals(final Object other) {

    if (other == null) {

        return false;

    }

    if (!(other instanceof Size)) {

        return false;

    }

```

```

    final Size otherSize = (Size) other;

    return (width == otherSize.width && height == otherSize.height);
}

```

```

@Override

public int hashCode() {

    return width * 32713 + height;
}

```

```

@Override

public String toString() {

    return dimensionsAsString(width, height);
}
}

```

Labelmap dataset

```

person
bicycle
car
motorcycle
airplane
bus
train
truck
boat
traffic light
fire hydrant
???
stop sign
parking meter
bench
bird
cat
dog
horse
sheep
cow
elephant
bear
zebra
giraffe
???
backpack
umbrella
???
???
handbag
tie

```

suitcase
frisbee
skis
snowboard
sports ball
kite
baseball bat
baseball glove
skateboard
surfboard
tennis racket
bottle
???
wine glass
cup
fork
knife
spoon
bowl
banana
apple
sandwich
orange
broccoli
carrot
hot dog
pizza
donut
cake
chair
couch
potted plant
bed
???
dining table
???
???
toilet
???
tv
laptop
mouse
remote
keyboard
cell phone
microwave
oven
toaster
sink
refrigerator
???
book
clock
vase
scissors
teddy bear
hair drier
toothbrush

DetectorTest

```
package org.tensorflow.lite.examples.detection;

import static com.google.common.truth.Truth.assertThat;

import static java.lang.Math.abs;

import static java.lang.Math.max;

import static java.lang.Math.min;


import android.content.res.AssetManager;

import android.graphics.Bitmap;

import android.graphics.Bitmap.Config;

import android.graphics.BitmapFactory;

import android.graphics.Canvas;

import android.graphics.Matrix;

import android.graphics.RectF;

import android.util.Size;

import androidx.test.ext.junit.runners.AndroidJUnit4;

import androidx.test.platform.app.InstrumentationRegistry;

import java.io.IOException;

import java.io.InputStream;

import java.util.ArrayList;

import java.util.List;

import java.util.Scanner;

import org.junit.Before;

import org.junit.Test;

import org.junit.runner.RunWith;

import org.tensorflow.lite.examples.detection.env.ImageUtils;

import org.tensorflow.lite.examples.detection.tflite.Detector;

import org.tensorflow.lite.examples.detection.tflite.Detector.Recognition;

import org.tensorflow.lite.examples.detection.tflite.TFLiteObjectDetectionAPIModel;

/** Golden test for Object Detection Reference app. */

@RunWith(AndroidJUnit4.class)
```

```

public class DetectorTest {

    private static final int MODEL_INPUT_SIZE = 300;

    private static final boolean IS_MODEL_QUANTIZED = true;

    private static final String MODEL_FILE = "detect.tflite";

    private static final String LABELS_FILE = "labelmap.txt";

    private static final Size IMAGE_SIZE = new Size(640, 480);


    private Detector detector;

    private Bitmap croppedBitmap;

    private Matrix frameToCropTransform;

    private Matrix cropToFrameTransform;


    @Before

    public void setUp() throws IOException {

        detector =

            TFLiteObjectDetectionAPIModel.create(

                InstrumentationRegistry.getInstrumentation().getContext(),

                MODEL_FILE,

                LABELS_FILE,

                MODEL_INPUT_SIZE,

                IS_MODEL_QUANTIZED);

        int cropSize = MODEL_INPUT_SIZE;

        int previewWidth = IMAGE_SIZE.getWidth();

        int previewHeight = IMAGE_SIZE.getHeight();

        int sensorOrientation = 0;

        croppedBitmap = Bitmap.createBitmap(cropSize, cropSize, Config.ARGB_8888);


        frameToCropTransform =

            ImageUtils.getTransformationMatrix(

                previewWidth, previewHeight,

                cropSize, cropSize,

                sensorOrientation, false);
    }
}

```

```

cropToFrameTransform = new Matrix();

frameToCropTransform.invert(cropToFrameTransform);
}

@Test

public void detectionResultsShouldNotChange() throws Exception {

    Canvas canvas = new Canvas(croppedBitmap);

    canvas.drawBitmap(loadImage("table.jpg"), frameToCropTransform, null);

    final List<Recognition> results = detector.recognizeImage(croppedBitmap);

    final List<Recognition> expected = loadRecognitions("table_results.txt");

    for (Recognition target : expected) {

        // Find a matching result in results

        boolean matched = false;

        for (Recognition item : results) {

            RectF bbox = new RectF();

            cropToFrameTransform.mapRect(bbox, item.getLocation());

            if (item.getTitle().equals(target.getTitle())

                && matchBoundingBoxes(bbox, target.getLocation())

                && matchConfidence(item.getConfidence(), target.getConfidence())) {

                matched = true;

                break;

            }

        }

        assertTrue(matched);

    }

}

// Confidence tolerance: absolute 1%

private static boolean matchConfidence(float a, float b) {

    return abs(a - b) < 0.01;

}

```

```

// Bounding Box tolerance: overlapped area > 90% of each one

private static boolean matchBoundingBoxes(RectF a, RectF b) {

float areaA = a.width() * a.height();

float areaB = b.width() * b.height();


RectF overlapped =

    new RectF(

        max(a.left, b.left), max(a.top, b.top), min(a.right, b.right), min(a.bottom, b.bottom));

float overlappedArea = overlapped.width() * overlapped.height();

return overlappedArea > 0.9 * areaA && overlappedArea > 0.9 * areaB;

}


private static Bitmap loadImage(String fileName) throws Exception {

    AssetManager assetManager =

        InstrumentationRegistry.getInstrumentation().getContext().getAssets();

    InputStream inputStream = assetManager.open(fileName);

    return BitmapFactory.decodeStream(inputStream);

}


// The format of result:

// category bbox.left bbox.top bbox.right bbox.bottom confidence

// ...

// Example:

// Apple 99 25 30 75 80 0.99

// Banana 25 90 75 200 0.98

// ...

private static List<Recognition> loadRecognitions(String fileName) throws Exception {

    AssetManager assetManager =

        InstrumentationRegistry.getInstrumentation().getContext().getAssets();

    InputStream inputStream = assetManager.open(fileName);

    Scanner scanner = new Scanner(inputStream);

    List<Recognition> result = new ArrayList<>();

    while (scanner.hasNext()) {

```

```

String category = scanner.next();

category = category.replace('_', ' ');

if (!scanner.hasNextFloat()) {

    break;

}

float left = scanner.nextFloat();

float top = scanner.nextFloat();

float right = scanner.nextFloat();

float bottom = scanner.nextFloat();

RectF boundingBox = new RectF(left, top, right, bottom);

float confidence = scanner.nextFloat();

Recognition recognition = new Recognition(null, category, confidence, boundingBox);

result.add(recognition);

}

return result;

}

```

5.01 CONCLUSIONS

Object detection has been used in many applications, with the most popular ones being:

- (i) human-computer interaction (HCI),
- (ii) robotics (e.g., service robots)
- (iii) consumer electronics (e.g., smart-phones)
- (iv) security (e.g., recognition, tracking),
- (v) retrieval (e.g., search engines, photo management)
- (vi) transportation (e.g., autonomous and assisted driving).

Each of these applications has different requirements, including: processing time (off-line, on-line, or real-time), robustness to occlusions, invariance to rotations (e.g., in-plane rotations), and detection under pose changes. While many applications consider the detection of a single object class (e.g., faces) and from a single view (e.g., frontal faces), others require the detection of multiple object classes (humans, vehicles, etc.), or of a single class from multiple views (e.g., side and frontal view of vehicles). In general, most systems can detect only a single object class from a restricted set of views and poses.

Utilising multiple base models and object detection frameworks, we successfully trained and implemented a variety of models that could be used for real-time detection of notes in an Android application. Using Transfer Learning, the network could be trained on a relatively small amount of data while still achieving high mAP scores. Our results conclude that ML models are viable for deployment on mobile devices in terms of inference time and accuracy and can provide a heuristic based corner-detection algorithm with bounding boxes of high recall. The results given in this report emphasises the possibility of using ML models and algorithms for object detection on mobile devices. It is shown that the trained models are feasible for use in a real-time environment on devices with limited computational power while remaining highly accurate and reliable for the end user. In order to further develop the work presented in this report careful gathering of diversified training data could lead to increased performance, as more images from environments typical to the end use case could increase the performance of the less complex models when detecting multiple notes, as these models might have been limited by the quality of

the training data. Furthermore, utilising Capsule Networks instead of CNNs as the base model for the network could extend the image vision capabilities of the model and add improved learning of part-whole relationships and viewpoint variation and thus increase the recall performance of the model.

5.02 SUMMARY

Object detection methods aim to identify all target objects in the target image and determine the categories and position information in order to achieve machine vision understanding. Numerous approaches have been proposed to solve this problem, mainly inspired by methods of computer vision and deep learning.

5.03 FUTURE WORK

The future of object detection technology is in the process of proving itself, and much like the original Industrial Revolution, it has the potential to free people from menial jobs that can be done more efficiently and effectively by machines. It will also open up new avenues of research and operations that will reap additional benefits in the future.

Thus, these challenges circumvent the need for a lot of training requiring a massive number of datasets to serve more nuanced tasks, with its continued evolution, along with the devices and techniques that make it possible, it could soon become the next big thing in the future.

REFERENCES

- [1] Mansour Ahmadi, Angelo Sotgiu, and Giorgio Giacinto. “IntelliAV: Toward the Feasibility of Building Intelligent Anti-malware on Android Devices”. In: Machine Learning and Knowledge Extraction. Ed. by Andreas Holzinger, Peter Kieseberg, Amin Tjoa, and Edgar Weippl. Cham: Springer International Publishing, 2017, pp. 137–154.
- [2] James Bergstra and Yoshua Bengio. “Random Search for Hyperparameter Optimization”. In: J. Mach. Learn. Res. 13 (Feb. 2012), pp. 281–305. ISSN: 1532-4435.
- [3] Christopher M. Bishop. Pattern Recognition and Machine Learning. Springer, 2006.
- [4] João Cartucho. mAP. <https://github.com/Cartucho/mAP>. Accessed: 2018-04-09.
- [5] Marc Claesen and Bart De Moor. “Hyperparameter Search in Machine Learning”. In: CoRR abs/1502.02127 (2015). arXiv: 1502.02127.
- [6] PASCAL2 Network of Excellence. Pascal VOC. <http://hostrobots.ox.ac.uk/pascal/VOC/>. Accessed: 2018-03-21.
- [7] Facebook. Instagram. <https://www.instagram.com/>. Accessed: 2018-03-20.
- [8] Sanja Fidler. Lecture Notes in Intro to Image Understanding.
- [9] Nicholas Frosst Geoffrey E Hinton Sara Sabour. “Matrix capsules with EM routing”. In: International Conference on Learning Representations (2018). accepted as poster.
- [10] Ross B. Girshick. “Fast R-CNN”. In: CoRR abs/1504.08083 (2015). arXiv: 1504.08083
- [11] StackOverflow
- [12] <https://www.tensorflow.org/lite/>